

Table of Contents

Table of Contents

XQuant：人人都是量化交易员	1.1
环境搭建 - Mac	2.1
环境搭建 - Windows	2.2
获取沪深300ETF历史数据	3.1
定投策略回测	3.2
基准对比	3.3
均线可视化	3.4
均线择时策略	3.5
参数扫描	3.6
过拟合识别	3.7
信号质量评估	4.1
GDP 锚定	4.2
相关性分析	4.3
等权重分配	5.1
风险平价	5.2
动量排名分配	5.3
调仓频率	6.1
止损策略	6.2
止盈策略	6.3
基准与指标	7.1
年度分解分析	7.2
收益分布与回撤	7.3
参数敏感性分析	7.4
参数优化	8.1
向前滚动验证	8.2
交叉验证	8.3
规则复杂度控制	8.4
订单生成器	9.1
成交价模拟	9.2
Alpaca 纸交易	9.3
执行报告	9.4
策略监控	10.1
诊断分析	10.2
迭代方法论	10.3
因子介绍	11.1
动量因子 IC 分析	11.2
因子到策略	11.3

XQuant Learning

《XQuant：人人都是量化交易员》课程配套的 Specs 与 Notebooks 开源仓库。

这是什么

本仓库包含课程中每个章节的：

- **Specs** — 任务说明书。学员通过撰写 spec 指挥 AI 完成所有编码工作，而非自己写代码。
- **Notebooks** — 可运行的 Jupyter Notebook 源代码，是每个 spec 的最终产出。

课程结构

课程围绕 9 个核心问题展开，从零构建一个可实盘运行的量化交易系统：

章节	问题	内容
Q0	开始前的准备	环境搭建
Q1	量化交易怎么赚钱？	建立直觉，跑通最小闭环
Q2	买什么？	标的筛选，构建 Universe
Q3	买多少？	权重分配，仓位管理
Q4	什么时候买卖？	信号生成，Agent 决策逻辑
Q5	怎么知道策略有效？	回测验证，建立评估体系
Q6	如何避免自欺欺人？	过拟合识别，稳健性检验
Q7	如何真正执行？	实盘系统，从回测到交易
Q8	如何持续改进？	策略迭代，建立方法论闭环
Q9	量化交易的日常是什么？	因子研究，从想法到策略的日常 workflows

目录结构

```
xquant-learning/
├─ q0-before-you-start/
│   └─ specs/
├─ q1-how-to-profit/
│   ├── specs/          # 6 个 spec : 获取数据 → 定投回测 → 基准对比 → 均线择时 → 参数扫描 → 过拟合
│   └─ notebooks/      # 01-minimal-strategy.ipynb
├─ q2-what-to-buy/
│   └─ ...
└─ ...
```

如何使用

1. 阅读某章节的 `specs/` 目录，了解每一步的任务目标
2. 将 spec 喂给 AI 编程工具（Claude Code / Cursor / ChatGPT 等），让 AI 帮你完成实现
3. 对照 `notebooks/` 中的参考实现，验证你的产出

课程特色

- **编程语言无关** — 全程用自然语言驱动 AI，学的是"如何描述问题"而非"如何写 Python"
- **问题驱动** — 不从理论开始，而从"这个策略能赚钱吗"开始
- **Agent 思维框架** — 把策略看作 Agent：Environment（标的池）→ State（市场状态）→ Action（买卖决策）

License

MIT

所有命令在沙箱外运行。

请帮我配置量化交易课程的 Python 编程环境。请严格按照以下步骤操作：

重要：本任务涉及安装系统工具和 Python 包，所有终端命令都需要在沙箱外（真实终端）运行。如果 TRAE 提示选择运行环境，请选择「在终端中运行」或「沙箱外运行」。

第零步：确认当前工作目录

我已经在 TRAE 中打开了一个文件夹作为课程学习目录。请先在终端运行 `pwd` 获取当前目录路径，后续所有操作都在这个目录下进行。

第一步：安装 uv

uv 是一个 Python 包管理工具，用它来管理 Python 版本和依赖包。

在终端运行：`curl -LsSf https://astral.sh/uv/install.sh | sh`

安装完成后，运行以下命令让 uv 生效：`source $HOME/.local/bin/env`

然后验证安装成功：`uv --version`

第二步：在当前目录创建虚拟环境

在当前目录下用 uv 创建虚拟环境（会自动下载 Python 3.12）：

```
uv venv --python 3.12
```

然后激活虚拟环境：`source .venv/bin/activate`

验证 Python 版本：`python --version`

应该显示 Python 3.12.x。

第三步：安装依赖包

使用清华镜像源安装以下 6 个包（国内速度更快）：

```
uv pip install --index-url https://pypi.tuna.tsinghua.edu.cn/simple jupyter pandas numpy matplotlib akshare yfinance
```

然后安装课程配套的量化交易框架 open-xquant（从 GitHub 安装，尚未发布到 PyPI）：

```
uv pip install "open-xquant @ git+https://github.com/xingwudao/open-xquant.git"
```

第四步：创建并运行环境检查脚本

请在当前目录下创建文件 check_env.py，内容如下：

```
""" XQuant 课程环境检查脚本
```

使用方法：在激活虚拟环境后运行 python check_env.py

本脚本检查量化交易课程所需的编程环境是否配置正确。 """

```
import sys import shutil
```

```
REQUIRED_PYTHON = (3, 12)
```

```
REQUIRED_PACKAGES = [ ("jupyter", "jupyter"), ("pandas", "pandas"), ("numpy", "numpy"), ("matplotlib", "matplotlib"),  
("akshare", "akshare"), ("yfinance", "yfinance"), ("oxq", "open-xquant"), ]
```

```
AKSHARE_TEST_FUNC = "fund_etf_hist_em" AKSHARE_TEST_KWARGS = {"symbol": "510300", "period": "daily",  
"adjust": "qfq"}
```

```
OPEN_XQUANT_INSTALL = 'uv pip install "open-xquant @ git+https://github.com/xingwudao/open-xquant.git'"
```

```
def check_python_version(): v = sys.version_info version_str = f"{v.major}.{v.minor}.{v.micro}" if (v.major, v.minor) >=  
REQUIRED_PYTHON: return True, f"Python {version_str}" return False, f"Python {version_str} ( 需要 >=  
{REQUIRED_PYTHON[0]}.{REQUIRED_PYTHON[1]} ) "
```

```
def check_venv(): in_venv = sys.prefix != sys.base_prefix if in_venv: return True, "虚拟环境已激活" return False, "未检测到  
虚拟环境，请先运行 source .venv/bin/activate"
```

```
def checkpackage(importname, display_name): try: __import__(import_name) return True, display_name except ImportError: if  
display_name == "open-xquant": return False, f"{display_name} — 未安装，请运行: {OPEN_XQUANT_INSTALL}" return  
False, f"{display_name} — 未安装，请运行: uv pip install {display_name}"
```

```
def check_jupyter_command(): if shutil.which("jupyter"): return True, "jupyter 命令可用" return False, "jupyter 命令不可  
用，请运行: uv pip install jupyter"
```

```
def check_data_source(): try: import akshare as ak func = getattr(ak, AKSHARE_TEST_FUNC) df =  
func(**AKSHARE_TEST_KWARGS) if len(df) > 0: return True, f"akshare 数据源正常 ( 获取到 {len(df)} 条数据 )" return  
False, "akshare 返回了空数据，可能是接口变动" except ImportError: return False, "akshare 未安装，跳过数据源测试"  
except Exception as e: return False, f"akshare 数据获取失败: {e}"
```

```
def main(): print() print("=" 50) print(" XQuant 课程环境检查") print("=" 50) print()
```

```

results = []

ok, msg = check_python_version()
results.append((ok, msg))

ok, msg = check_venv()
results.append((ok, msg))

for import_name, display_name in REQUIRED_PACKAGES:
    ok, msg = check_package(import_name, display_name)
    results.append((ok, msg))

ok, msg = check_jupyter_command()
results.append((ok, msg))

ok, msg = check_data_source()
results.append((ok, msg))

passed = 0
failed = 0
for ok, msg in results:
    if ok:
        print(f" [OK]   {msg}")
        passed += 1
    else:
        print(f" [FAIL] {msg}")
        failed += 1

print()
print("-" * 50)

if failed == 0:
    print(f" 结果: {passed}/{passed} 全部通过!")
    print()
    print(" 环境配置完成, 可以开始课程了。")
else:
    print(f" 结果: {passed}/{passed + failed} 通过, {failed} 项需要修复")
    print()
    print(" 请把上面的输出结果发给 AI 助手, 让它帮你修复失败项。")

print()
return 0 if failed == 0 else 1

```

if **name** == "**main**": sys.exit(main())

文件创建完成后, 运行: python check_env.py

如果所有检查项都显示 [OK], 环境配置就完成了。如果有失败项, 请把输出结果发给我, 我帮你排查。

注意事项

- 如果终端提示 `uv: command not found`，请先运行 `source $HOME/.local/bin/env`
- 如果 `pip install` 很慢或超时，已经指定了清华镜像源，应该不会有问题
- 每次打开新的终端窗口，都需要先激活虚拟环境：`source .venv/bin/activate`

所有命令在沙箱外运行。

请帮我配置量化交易课程的 Python 编程环境。我用的是 Windows。请严格按照以下步骤操作：

重要：本任务涉及安装系统工具和 Python 包，所有终端命令都需要在沙箱外（真实终端）运行。如果 TRAE 提示选择运行环境，请选择「在终端中运行」或「沙箱外运行」。

第零步：确认当前工作目录

我已经在 TRAE 中打开了一个文件夹作为课程学习目录。请先在终端运行 `cd` 或 `echo %cd%` 获取当前目录路径，后续所有操作都在这个目录下进行。

第一步：安装 uv

uv 是一个 Python 包管理工具，用它来管理 Python 版本和依赖包。

在 PowerShell 中运行：`powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"`

安装完成后，关闭并重新打开终端（让 PATH 生效），然后验证安装成功：`uv --version`

第二步：在当前目录创建虚拟环境

在当前目录下用 uv 创建虚拟环境（会自动下载 Python 3.12）：

```
uv venv --python 3.12
```

然后激活虚拟环境：`.venv\Scripts\activate`

验证 Python 版本：`python --version`

应该显示 Python 3.12.x。

第三步：安装依赖包

使用清华镜像源安装以下 6 个包（国内速度更快）：

```
uv pip install --index-url https://pypi.tuna.tsinghua.edu.cn/simple jupyter pandas numpy matplotlib akshare yfinance
```

然后安装课程配套的量化交易框架 open-xquant（从 GitHub 安装，尚未发布到 PyPI）：

```
uv pip install "open-xquant @ git+https://github.com/xingwudao/open-xquant.git"
```

第四步：创建并运行环境检查脚本

请在当前目录下创建文件 check_env.py，内容如下：

```
""" XQuant 课程环境检查脚本
```

使用方法：在激活虚拟环境后运行 python check_env.py

本脚本检查量化交易课程所需的编程环境是否配置正确。 """

```
import sys import shutil
```

```
REQUIRED_PYTHON = (3, 12)
```

```
REQUIRED_PACKAGES = [ ("jupyter", "jupyter"), ("pandas", "pandas"), ("numpy", "numpy"), ("matplotlib", "matplotlib"),  
 ("akshare", "akshare"), ("yfinance", "yfinance"), ("oxq", "open-xquant"), ]
```

```
AKSHARE_TEST_FUNC = "fund_etf_hist_em" AKSHARE_TEST_KWARGS = {"symbol": "510300", "period": "daily",  
 "adjust": "qfq"}
```

```
OPEN_XQUANT_INSTALL = 'uv pip install "open-xquant @ git+https://github.com/xingwudao/open-xquant.git'"
```

```
def check_python_version(): v = sys.version_info version_str = f"{v.major}.{v.minor}.{v.micro}" if (v.major, v.minor) >=  
REQUIRED_PYTHON: return True, f"Python {version_str}" return False, f"Python {version_str} ( 需要 >=  
{REQUIRED_PYTHON[0]}.{REQUIRED_PYTHON[1]} ) "
```

```
def check_venv(): in_venv = sys.prefix != sys.base_prefix if in_venv: return True, "虚拟环境已激活" return False, "未检测到  
虚拟环境，请先运行 .venv\Scripts\activate"
```

```
def checkpackage(importname, display_name): try: __import__(import_name) return True, display_name except ImportError: if  
display_name == "open-xquant": return False, f"{display_name} — 未安装，请运行: {OPEN_XQUANT_INSTALL}" return  
False, f"{display_name} — 未安装，请运行: uv pip install {display_name}"
```

```
def check_jupyter_command(): if shutil.which("jupyter"): return True, "jupyter 命令可用" return False, "jupyter 命令不可  
用，请运行: uv pip install jupyter"
```

```
def check_data_source(): try: import akshare as ak func = getattr(ak, AKSHARE_TEST_FUNC) df =  
func(**AKSHARE_TEST_KWARGS) if len(df) > 0: return True, f"akshare 数据源正常 ( 获取到 {len(df)} 条数据 )" return  
False, "akshare 返回了空数据，可能是接口变动" except ImportError: return False, "akshare 未安装，跳过数据源测试"  
except Exception as e: return False, f"akshare 数据获取失败: {e}"
```

```
def main(): print() print("=" 50) print(" XQuant 课程环境检查") print("=" 50) print()
```

```

results = []

ok, msg = check_python_version()
results.append((ok, msg))

ok, msg = check_venv()
results.append((ok, msg))

for import_name, display_name in REQUIRED_PACKAGES:
    ok, msg = check_package(import_name, display_name)
    results.append((ok, msg))

ok, msg = check_jupyter_command()
results.append((ok, msg))

ok, msg = check_data_source()
results.append((ok, msg))

passed = 0
failed = 0
for ok, msg in results:
    if ok:
        print(f" [OK]   {msg}")
        passed += 1
    else:
        print(f" [FAIL] {msg}")
        failed += 1

print()
print("-" * 50)

if failed == 0:
    print(f" 结果: {passed}/{passed} 全部通过!")
    print()
    print(" 环境配置完成, 可以开始课程了。")
else:
    print(f" 结果: {passed}/{passed + failed} 通过, {failed} 项需要修复")
    print()
    print(" 请把上面的输出结果发给 AI 助手, 让它帮你修复失败项。")

print()
return 0 if failed == 0 else 1

```

if **name** == "**main**": sys.exit(main())

文件创建完成后, 运行: python check_env.py

如果所有检查项都显示 [OK], 环境配置就完成了。如果有失败项, 请把输出结果发给我, 我帮你排查。

注意事项

- 如果提示 uv 不是可识别的命令，请关闭终端重新打开
- 如果 pip install 很慢或超时，已经指定了清华镜像源，应该不会有问题
- 每次打开新的终端窗口，都需要先激活虚拟环境：`.venv\Scripts\activate`

Spec: 获取沪深300ETF历史数据

所有命令在沙箱外运行。

上下文

学员已完成环境配置（Python 3.12 + 虚拟环境 + 依赖包已安装）。这是课程的第一个操作步骤。

任务

1. 在当前工作目录下创建 Jupyter Notebook 文件 `q1-strategy.ipynb` 并打开它
2. 在 notebook 中创建代码单元格，获取沪深300ETF最近5年的历史日线数据并可视化

要求

1. 导入所需库：yfinance, pandas, numpy, matplotlib.pyplot
2. 设置 matplotlib 中文显示支持：
 - macOS 字体优先使用 Arial Unicode MS，备选 STHeiti
 - Windows 字体使用 SimHei
 - 设置 `axes.unicode_minus = False` 解决负号显示问题
3. 使用 yfinance 获取数据：
 - 标的代码：510300.SS（上交所沪深300ETF）
 - 起始日期：2021-01-01（获取最近约5年数据），不指定结束日期（默认到最新）
 - 传入参数 `auto_adjust=True, multi_level_index=False` 避免警告和多级列名
4. 将数据存储在变量 `df` 中
5. 确保索引为 `DatetimeIndex`，列名包含 `Close`

结果呈现

- 1. 打印数据的前 5 行
- 2. 打印数据时间范围（起始日期 到 结束日期）和总行数
- 3. 画一条收盘价（Close）折线图，标题为「沪深300ETF (510300) 收盘价走势」

Spec: 实现简单定投回测

所有命令在沙箱外运行。

上下文

在 `q1-strategy.ipynb` 中已有上一步的代码：使用 `yfinance` 获取了沪深300ETF（510300.SS）的日线数据，存储在变量 `df` 中，索引为 `DatetimeIndex`，包含 `Close` 列。

任务

基于已有数据，实现最简单的定投策略回测：每月第一个交易日，用固定金额买入。

要求

1. 在 notebook 中新建代码单元格，基于已有的 `df` 变量
2. 策略规则：每月第一个交易日，用 1000 元按收盘价买入
3. 将回测逻辑封装为函数 `backtest_dca(df, monthly_amount=1000)`，返回 DataFrame（后续步骤会复用此函数）
4. 实现回测逻辑：
 - 按月分组，取每月第一个交易日作为买入日
 - 每月买入份额 = $1000 / \text{当日收盘价}$
 - 累计持有份额 = 逐月累加
 - 累计投入成本 = $\text{月数} \times 1000$
 - 每月末持仓市值 = $\text{累计份额} \times \text{当月最后一个交易日收盘价}$
 - 累计收益率 = $(\text{持仓市值} - \text{累计成本}) / \text{累计成本}$
5. 调用函数得到结果：`result_dca = backtest_dca(df)`，包含列：`total_cost`, `total_shares`, `portfolio_value`, `return`

结果呈现

1. 打印：投资期间、投资月数、总投入金额、最终市值、最终收益率
2. 画两张图（上下排列，figsize 12×8）：
 - 图1「持仓市值 vs 累计成本」：两条线 + 盈利区域填充绿色、亏损区域填充红色
 - 图2「累计收益率」：收益率曲线，零线上方填充绿色、下方填充红色

Spec: 和基准对比

所有命令在沙箱外运行。

上下文

在 `q1-strategy.ipynb` 中已有：

- 沪深300ETF 日线数据（变量 `df`，含 Close 列）
- 简单定投回测结果（变量 `result_dca`，含 return 列）

任务

计算「一次性买入持有」作为基准，和定投策略对比，帮助理解收益来源。

要求

1. 在 notebook 中新建代码单元格
2. 基准策略 (Buy & Hold) : 在数据起始日用全部资金一次性买入, 持有到最后一天
3. 基准收益率 = (终点收盘价 - 起点收盘价) / 起点收盘价, 存储在变量 `benchmark_return` 中
4. 计算基准的逐月累计收益率曲线 (每月末收盘价相对起始价的涨跌幅), 用于可视化对比
5. 与定投策略的最终收益率对比

结果呈现

1. 打印：沪深300ETF 起点价格、终点价格
2. 打印：一次性买入收益率、定投收益率、两者差异
3. 根据对比结果打印一句话说明：
 - 如果定投跑赢：「在下跌市中，定投通过分批买入避开了部分高位」
 - 如果定投跑输：「在上涨市中，越早全部投入越好」
4. 无论哪种结果，都补充一句：「但两种策略都没有 Alpha——收益完全取决于市场涨跌，这就是 Beta」
5. 画一张对比图（figsize 12×6）：定投累计收益率曲线 vs 一次性买入累计收益率曲线，图例标注两种策略名称

Spec: 可视化均线与交叉信号

所有命令在沙箱外运行。

上下文

在 `q1-strategy.ipynb` 中已有：

- 沪深300ETF 日线数据（变量 `df`，含 Close 列）

上一节引入了均线的概念——过去 N 天收盘价的平均值。接下来要画一张图，让你亲眼看到均线长什么样、交叉发生在哪里。

任务

在 notebook 中新建代码单元格，画出收盘价与三条均线的对比图，并标记交叉点。

要求

1. 在 notebook 中新建代码单元格
2. 截取最近 1 年数据，让图更清晰：

```
one_year_ago = df.index[-1] - pd.DateOffset(years=1)
df_recent = df.loc[one_year_ago:].copy()
```

3. 基于 `df_recent` 计算三条均线：

```
ma10 = df_recent['Close'].rolling(10).mean()
ma20 = df_recent['Close'].rolling(20).mean()
ma60 = df_recent['Close'].rolling(60).mean()
```

4. 找出收盘价与 MA20 的交叉点：

- 上穿：前一天 $\text{Close} \leq \text{MA20}$ ，当天 $\text{Close} > \text{MA20}$
- 下穿：前一天 $\text{Close} \geq \text{MA20}$ ，当天 $\text{Close} < \text{MA20}$

5. 画一张对比图（figsize 14×7）：

- 收盘价（黑色实线，标注"收盘价"）
- MA10（绿色虚线，标注"10日均线"）
- MA20（蓝色实线，标注"20日均线"）
- MA60（红色点线，标注"60日均线"）
- 上穿点：绿色向上三角标记（`marker='^', markersize=10`）
- 下穿点：红色向下三角标记（`marker='v', markersize=10`）
- 标题「沪深300ETF：收盘价与均线（最近1年）」
- 图例 + 网格

结果呈现

- 1. 价格与均线对比图（含交叉点标记）
- 2. 打印交叉统计：

MA20 交叉统计（最近1年）：
上穿（买入信号）：N 次
下穿（卖出信号）：N 次

Spec: 加入均线择时条件

所有命令在沙箱外运行。

上下文

在 `q1-strategy.ipynb` 中已有：

- 沪深300ETF 日线数据（变量 `df`，含 `Close` 列）
- 简单定投回测函数 `backtest_dca` 和结果（`result_dca`）
- 基准对比代码

任务

在定投策略基础上加入均线择时条件，尝试获取超越市场的收益（Alpha）。

要求

1. 在 notebook 中新建代码单元格
2. 新策略规则：每月第一个交易日，**仅当收盘价 > N日均线时**买入 1000 元，否则跳过该月
3. 实现函数 `backtest_dca_with_ma(df, monthly_amount=1000, ma_period=20)`，返回格式与 `result_dca` 一致，额外包含 `signal` 列（布尔值，是否满足买入条件）。后续步骤会复用此函数
4. 注意处理：
 - 均线计算初期（前 N 天）没有均线值，这些月份不买入
 - 跳过买入的月份，累计成本和份额保持不变
 - 计算收益率时，累计成本为 0 的月份（尚未发生任何买入）收益率直接设为 0，不要用除法，避免除零警告
5. 分别用 MA10、MA20、MA60 三个参数运行回测

结果呈现

1. 打印每个参数的结果，格式：`MA10: 收益率 X.XX%, 买入 N/M 次`
2. 打印简单定投收益率作为对照
3. 画一张对比图（`figsize 12×6`）：
 - 简单定投收益率曲线（灰色虚线）
 - MA10、MA20、MA60 三条收益率曲线（不同颜色实线）
 - 图例 + 网格 + 零线

Spec: 参数扫描找最优

所有命令在沙箱外运行。

上下文

在 `q1-strategy.ipynb` 中已有：

- 沪深300ETF 日线数据（变量 `df`）
- 简单定投回测结果（`result_dca`）
- 带均线条件的回测函数 `backtest_dca_with_ma`（可传入 `ma_period` 参数）

任务

扫描所有均线参数（MA5 到 MA120），找到历史收益率最高的「最优参数」并可视化。

要求

1. 在 notebook 中新建代码单元格
2. 扫描范围：MA5 到 MA120，步长为 5（共 24 个参数）
3. 对每个参数调用已有的均线回测函数，记录最终收益率
4. 找出收益率最高的参数和对应收益率

结果呈现

1. 打印：最优参数、最优收益率
2. 打印：简单定投收益率作为对照、收益提升幅度
3. 画一张柱状图（figsize 12×5）：
 - X轴：均线周期（天），Y轴：收益率
 - 正收益的柱子绿色，负收益的柱子红色
 - 最优参数的柱子用金色高亮、黑色边框
 - 灰色虚线标注简单定投的收益率水平
 - 黑色实线标注零线
 - 图例标注最优参数编号

Spec: 样本外测试

所有命令在沙箱外运行。

上下文

在 `q1-strategy.ipynb` 中已有：

- 沪深300ETF 日线数据（变量 `df`）
- 简单定投回测函数 `backtest_dca`
- 带均线条件的回测函数 `backtest_dca_with_ma`（可传入 `ma_period` 参数）
- 参数扫描代码（扫描 MA5 到 MA120）

任务

将数据分为训练集和测试集，验证「最优参数」在未见过的数据上是否仍然有效。

要求

1. 在 notebook 中新建代码单元格
2. 数据分割：按时间前 60% 为训练集，后 40% 为测试集
3. 在训练集上扫描参数（MA5 到 MA120，步长 5），找到训练集上的最优参数
4. 用该最优参数在测试集上运行均线回测
5. 同时在测试集上运行简单定投作为对照
6. 对比三个收益率：训练集最优、测试集该参数、测试集简单定投

结果呈现

1. 打印：训练集时间范围和数据条数、测试集时间范围和数据条数
2. 打印：训练集最优参数和收益率
3. 打印：测试集上该参数的收益率
4. 打印：测试集上简单定投的收益率
5. 对比结论：
 - 如果最优参数在测试集上比简单定投还差：打印「这就是过拟合——你在历史数据上找到的是巧合，不是规律」
 - 否则：打印「结果看起来还行，但仍需谨慎，样本量有限」

Spec: 个股 vs ETF——信号质量对比

所有命令在沙箱外运行。

上下文

本章探讨"该选哪些标的"。第一步：先拿大家熟悉的个股试试 Q1 学的均线策略，看看效果如何。

任务

在 notebook `q2-what-to-buy.ipynb` 中新建代码单元格，对少量 A 股个股跑均线策略并和沪深300ETF 对比，让学员直观感受个股信号的嘈杂。

要求

1. 在 notebook 中新建代码单元格

2. 导入所需库并设置中文显示：

- o `from oxq.core import Engine, Strategy`
- o `from oxq.data import YFinanceDownloader, LocalMarketDataProvider`
- o `from oxq.indicators import SMA`
- o `from oxq.signals import Crossover`
- o `from oxq.rules import ExitRule`
- o `from oxq.portfolio.optimizers import EqualWeightOptimizer`
- o `from oxq.trade import SimBroker`
- o `from oxq.universe import StaticUniverse`
- o `pandas, matplotlib.pyplot`
- o 设置 matplotlib 中文显示支持 (macOS: Arial Unicode MS / STHeiti, Windows: SimHei, `axes.unicode_minus = False`)

3. 使用 `YFinanceDownloader` 和 `LocalMarketDataProvider`

4. 下载沪深300ETF (`510300.SS`) 最近 3 年数据 (起始日期为 3 年前, 结束日期为当天)

5. 下载 5 只 A 股标的最近 3 年数据: [`"600519.SS"`, `"000858.SZ"`, `"601318.SS"`, `"000001.SZ"`, `"600036.SS"`] (茅台、五粮液、中国平安、平安银行、招商银行)

6. 对每个标的 (5 只个股 + 沪深300ETF), 构建一个 Strategy 并用 Engine 运行回测:

- o 信号: `Crossover(fast="sma_1", slow="sma_20")` ——SMA(1) 上穿 SMA(20) 等价于收盘价上穿 20 日均线
- o 将指标挂载到信号上: 通过 `signal.required_indicators` 指定 `sma_1` 对应 `SMA()` 参数 `{"column": "close", "period": 1}`, `sma_20` 对应 `SMA()` 参数 `{"column": "close", "period": 20}`
- o 构建策略: `Strategy(signals={...}, portfolio=EqualWeightOptimizer())`, 入场由 `EqualWeightOptimizer` 处理
- o 出场规则通过 `Engine.run(rules=[ExitRule(fast="sma_1", slow="sma_20")])` 传入
- o `router` 和 `receiver` 使用同一个 `SimBroker()` 实例
- o 用 `result.total_return()` 获取收益率
- o 用 `result.annualized_volatility()` 获取年化波动率

7. 画两张横向柱状图 (上下排列, `figsize 10×8`):

- o 第一张: 收益率对比, 沪深300ETF 用蓝色高亮, 其余按正负区分绿色/红色
- o 第二张: 年化波动率对比, 沪深300ETF 用蓝色, 个股用灰色

结果呈现

- 1. 打印每只标的的回测结果，格式：`600519.SS (贵州茅台): 收益率 X.XX% 年化波动率 X.XX%`
- 2. 两张横向柱状图（收益率 + 年化波动率）
- 3. 打印分析：
 - 「同样的均线策略，5 只个股的结果天差地别——从大幅盈利到严重亏损都有。」
 - 「再看波动率：个股年化波动率普遍在 XX%-XX%，而沪深300ETF 只有 XX%——300 只股票的噪音互相抵消，波动被分散掉了。」
 - 「组合能对抗波动。ETF 就是一种组合。」

Spec: 用 GDP 数据锁定投资方向

所有命令在沙箱外运行。

上下文

在 `q2-what-to-buy.ipynb` 中已有：

- Step 1 的实验代码：个股 vs ETF 信号质量对比
- 学员已理解组合能对抗波动，选择用 ETF 而非个股

当前问题：全球 ETF 有几百只，怎么缩小范围？

任务

在 notebook 中新建代码单元格，展示全球主要经济体的 GDP 数据，用数据支撑"锁定中美"的选择，并获取对应 ETF 的历史价格数据。

要求

1. 在 notebook 中新建代码单元格
2. 使用 `WorldBankDownloader` 一次性下载全球前 10 大经济体的 GDP 数据：

```
from oxq.data import WorldBankDownloader, read_factor
wb = WorldBankDownloader()
countries = ["USA", "CHN", "DEU", "JPN", "IND", "GBR", "FRA", "ITA", "BRA", "CAN"]
wb.download("gdp", countries=countries, start_year=2020, end_year=2024)
```

注意：第一个参数是指标名 `"gdp"`，`countries` 是列表，一次调用下载所有国家。

3. 使用 `read_factor` 读取数据，取最新一年，转换为万亿美元：

```
gdp_raw = read_factor("gdp", countries=countries, start_year=2020, end_year=2024)
latest_year = gdp_raw.index.max()
gdp_latest = gdp_raw.loc[latest_year] / 1e12
```

4. 画一张横向柱状图 (figsize 10×6)：
 - 按 GDP 从高到低排列
 - 中国 and 美国的柱子用红色和蓝色高亮，其余灰色
 - 在柱子右侧标注具体数值
 - 标题「全球前 10 大经济体 GDP」
5. 打印分析说明：中美两国 GDP 远超其他国家，合计占全球 GDP 的约 40%
6. 使用 `open-xquant` 下载两个对应的 ETF 最近 5 年数据 (起始日期 `2021-01-01`，结束日期为当天)：
 - `YFinanceDownloader` 下载 `510300.SS` (沪深300ETF, 代表中国核心资产)
 - `YFinanceDownloader` 下载 `513100.SS` (纳指100ETF, A股上市, 跟踪纳斯达克100指数)
7. 使用 `LocalMarketDataProvider().get_bars(symbol, start, end)` 读取数据，画出两只 ETF 的归一化价格走势对比图 (figsize 12×6)：
 - 将两只 ETF 的收盘价都归一化到起点 = 100
 - 两条线不同颜色，图例标注中文名称
 - 标题「沪深300 vs 纳斯达克100 归一化走势」

结果呈现

1. GDP 横向柱状图
2. 打印：「最直觉的思路：经济最强的国家，资本市场应该最有潜力。」
3. 打印：「数据说了算：锁定中国（沪深300 ETF）和美国（纳斯达克100 ETF）」
4. 两只 ETF 归一化走势对比图
5. 打印：「但一个新问题出现了——如果中美股市同时下跌怎么办？」

Spec: 相关性分析与投资宇宙构建

所有命令在沙箱外运行。

上下文

在 `q2-what-to-buy.ipynb` 中已有：

- 沪深300ETF (510300.SS) 和纳指100ETF (513100.SS) 的历史数据
- 学员已通过 GDP 数据锁定中美
- 当前问题：中美都是股市，可能同涨同跌，加什么能保护组合？

任务

在 notebook 中新建代码单元格，引入黄金 ETF 作为对冲资产，通过相关性分析验证三资产的分散化效果，并用 openxquant 的 StaticUniverse 正式构建投资宇宙。

要求

1. 在 notebook 中新建代码单元格
2. 导入 `numpy`（年化波动率计算需要）
3. 使用 `YFinanceDownloader` 下载黄金ETF（518880.SS）最近 5 年数据（起始日期 `2021-01-01`，结束日期为当天）
4. 使用 `LocalMarketDataProvider().get_bars(symbol, start, end)` 读取三只 ETF 的数据（510300.SS、513100.SS、518880.SS）
5. 将三只 ETF 的收盘价合并为一个 DataFrame，列名改为中文：`{"510300.SS": "沪深300", "513100.SS": "纳指100", "518880.SS": "黄金"}`
6. 计算日收益率（`pct_change`），然后计算相关性矩阵
7. 画相关性热力图（`figsize 8×6`）：
 - 使用 `matplotlib` 的 `imshow`
 - 在每个格子中标注相关系数数值（保留 2 位小数）
 - 使用 `coolwarm` 色系
 - 标题「三资产日收益率相关性」
8. 对比实验——组合 vs 单押：
 - 单押方案：100% 沪深300
 - 等权组合：沪深300、纳指100、黄金各 1/3
 - 计算两种方案的日收益率序列
 - 计算并对比：累计收益率、年化波动率、最大回撤
9. 画两条累计收益率曲线对比图（`figsize 12×6`）：
 - 单押沪深300（灰色虚线）vs 等权三资产组合（实线）
 - 图例标注方案名称
 - 标题「单押沪深300 vs 三资产等权组合」
10. 使用 `open-xquant` 构建投资宇宙：

```
from oxq.universe import StaticUniverse
universe = StaticUniverse(
    symbols=["510300.SS", "513100.SS", "518880.SS"],
    name="global-macro-etf"
)
snapshot = universe.get_universe(as_of_date="2026-02-28")
```

结果呈现

- 1. 相关性热力图
- 2. 打印相关性解读：
 - 「沪深300 与 纳指100 的相关性：X.XX —— 两个股票市场有一定联动，但并非完全同步」
 - 「沪深300 与 黄金 的相关性：X.XX —— 黄金与股市低相关，股市跌的时候它不一定跌」
- 3. 打印组合 vs 单押的对比表：

指标	单押沪深300	三资产等权组合
累计收益率	X.XX%	X.XX%
年化波动率	X.XX%	X.XX%
最大回撤	X.XX%	X.XX%

- 1. 累计收益率对比图
- 2. 打印关键认知：「数据说了算：分散风险的关键不是多买几个，而是买涨跌不同步的。」
- 3. 打印投资宇宙构建结果：「投资宇宙构建完成：{snapshot.symbols}」

Spec: 等权组合——最简单的"各买多少"

所有命令在沙箱外运行。

上下文

本章探讨"买多少"。Q2 选好了 3 只 ETF（沪深300、纳指100、黄金），现在要回答：每只各买多少？

任务

在 notebook `q3-how-much.ipynb` 中创建代码，跑等权组合回测，并展示三只 ETF 各自的表现差异。

要求

1. 获取当前工作目录，以此为根目录操作。
2. 阅读以下 oxq 模块的源码，了解各接口的输入、输出和参数含义：

- `oxq.portfolio.optimizers.EqualWeightOptimizer`
- `oxq.signals.Threshold`
- `oxq.indicators.RollingVolatility`
- `oxq.core.Engine`、`oxq.core.Strategy`
- `oxq.rules.RebalanceFrequencyRule`
- `oxq.trade.SimBroker`
- `oxq.data.YFinanceDownloader`、`oxq.data.LocalMarketDataProvider`
- `oxq.universe.StaticUniverse`

3. 创建 notebook `q3-how-much.ipynb`，导入所需库并设置中文显示（macOS: Arial Unicode MS / STHeiti, Windows: SimHei, `axes.unicode_minus = False`）。

4. 定义常量和标的中文名映射：

```
SYMBOLS = ("510300.SS", "513100.SS", "518880.SS")
SYMBOL_NAMES = {
    "510300.SS": "沪深300ETF",
    "513100.SS": "纳指100ETF",
    "518880.SS": "黄金ETF",
}
START = "2021-01-01"
```

5. 使用 `YFinanceDownloader` 下载 3 只 ETF 数据（起始日期 `2021-01-01`，结束日期为当天）。

6. 构建投资宇宙：

```
universe = StaticUniverse(symbols=SYMBOLS, name="global-macro-etf")
```

7. 使用 `LocalMarketDataProvider` 加载数据，并为每只 ETF 预计算 20 日滚动波动率（后面 spec-02 会用到）。
8. 用 `EqualWeightOptimizer` 构建组合优化器。使用 `Threshold` 信号作为"始终持有"的入场信号（设置 `signal.required_indicators` 包含所需指标）。打印最新一天每只 ETF 的权重值。
9. 打印权重的具体含义——假如总资金 10 万元，按权重换算每只各买多少钱。末尾加一句解释：权重（Weight）就是"这笔钱怎么分"——每只标的分到总资金的百分之多少。
10. 对三只 ETF 分别计算**单独持有**的表现指标（累计收益率、年化波动率、最大回撤），用于与组合对比。
11. 组装等权策略并运行回测。策略参数：
 - 使用 `Threshold` 信号（`column="close", threshold=0, relationship="gt"`）作为入场信号
 - 指标包含 20 日滚动波动率（通过 `signal.required_indicators` 设置）
 - 组合优化器使用 `EqualWeightOptimizer()`
 - 再平衡规则 `RebalanceFrequencyRule(interval_days=10)` 传入 `Engine.run(rules=[...])`
 - 权重时间序列通过 `result.weights_df()` 提取
12. 画净值曲线图（`figsize 12×6`）：
 - 三只 ETF 单独持有的净值线（灰色，用不同线型区分：虚线、点线、点划线）
 - 等权组合的净值线（蓝色实线，加粗）
 - 全部归一化到起点 = 100

- 标题「各买三分之一：等权组合 vs 单只持有」

结果呈现

1. 打印权重分配 + 10 万元换算

2. 打印对比表：

标的	累计收益率	年化波动率	最大回撤	沪深300 ETF	X.XX%	X.XX%
X.XX%	纳指100 ETF	X.XX%	X.XX%	黄金 ETF	X.XX%	X.XX%
X.XX%	等权组合	X.XX%	X.XX%			

3. 净值曲线对比图

4. 打印分析：

- 「三只 ETF 各自的年化波动率从 X.XX% 到 X.XX%，差了 N 倍。」
- 「但等权信号说：不管你多颠簸，都买一样多的钱。波动最大的那只，虽然只花了 1/3 的钱，却主导了组合的涨跌。」
- 「有没有更聪明的分法——比如，波动大的少买点？」

验证

执行成功的标志：

- notebook 中所有单元格无报错运行完毕
- 打印出 3 只 ETF 的等权权重（每只约 33.3%）和 10 万元换算
- 对比表包含 4 行（3 只 ETF + 等权组合），3 列指标均为合理数值（非 NaN、非 0）
- 净值曲线图显示 4 条线（3 条灰色线用不同线型区分 + 1 条蓝色实线）

Spec: 波动大的少买点——风险平价

所有命令在沙箱外运行。

上下文

在 `q3-how-much.ipynb` 中已有：

- 三只 ETF 的等权组合回测结果
- 学员已看到三只 ETF 波动率差异很大，等权分配让波动大的 ETF 主导了组合涨跌

当前问题：能不能按波动来分——波动大的少买，波动小的多买？

任务

在 notebook 中新建代码单元格，用风险平价信号运行回测，对比等权组合。

要求

1. 阅读以下 oxq 模块的源码，了解接口的输入、输出和参数含义：

- o `oxq.portfolio.optimizers.RiskParityOptimizer`
- o `oxq.signals.Threshold`（若 spec-01 未阅读）

2. 用 `RiskParityOptimizer` 构建组合优化器，配合 `Threshold` 信号计算权重（使用 spec-01 中已预计算的 `vol` 列），打印最新一天与等权权重的对比。

3. 打印风险平价权重的 10 万元换算，对比等权。

4. 提取 spec-01 中等权策略与风险平价策略的公共部分为 `COMMON` 字典，使两个策略只在 `portfolio` 优化器一行不同（信号均使用 `Threshold`）。组装风险平价策略（`portfolio=RiskParityOptimizer()`）并运行回测。权重时间序列通过 `result.weights_df()` 提取。

5. 打印两种方案的指标对比表（累计收益率、年化波动率、最大回撤）。

6. 画两条净值曲线对比图（`figsize 12×6`）：

- o 等权组合（灰色虚线）
- o 风险平价组合（蓝色实线）
- o 归一化到起点 = 100
- o 标题「等权 vs 风险平价」

7. 画风险平价的权重变化历史图（`figsize 12×4`）：

- o 堆叠面积图，展示三只 ETF 权重随时间的变化
- o 标题「风险平价：权重如何随波动率调整」

结果呈现

1. 权重对比表 + 10 万元换算
2. 指标对比表
3. 净值曲线对比图
4. 权重变化历史图
5. 打印分析：
 - 「波动大的少买点、波动小的多买点——组合波动率从 X.XX% 降到 X.XX%，最大回撤从 X.XX% 收窄到 X.XX%。」
 - 「这种方法叫风险平价（Risk Parity）——让每个资产对组合波动的贡献一样大。桥水基金的'全天候策略'用的就是这个思路。」

验证

执行成功的标志：

- 新增的单元格无报错运行完毕
- 权重对比表中风险平价权重三只 ETF 不相等（波动低的权重更高）
- 指标对比表中风险平价组合的年化波动率 $\leq$ 等权组合
- 净值曲线图显示 2 条线（灰色虚线 + 蓝色实线）
- 权重历史堆叠面积图显示三只 ETF 权重随时间变化

Spec: 涨得好的多买点——动量排名

所有命令在沙箱外运行。

上下文

在 `q3-how-much.ipynb` 中已有：

- 等权组合和风险平价组合的回测结果
- 学员已理解：等权不看任何指标，风险平价只看波动率

当前问题：风险平价更稳了，但它完全不看涨跌。能不能让涨得好的多买点？

任务

在 notebook 中新建代码单元格，用动量排名信号运行回测，与前两种方案三方对比。

要求

1. 阅读以下 oxq 模块的源码，了解接口的输入、输出和参数含义：

- `oxq.portfolio.optimizers.TopNRankingOptimizer`
- `oxq.indicators.Momentum`
- `oxq.indicators.Ratio`
- `oxq.signals.Threshold`（若前面 spec 未阅读）

2. 为每只 ETF 预计算 20 日动量，并用 `Ratio` 计算风险调整动量（ $ram = mom / vol$ ）。

3. 用 `TopNRankingOptimizer` 构建组合优化器（score 使用 ram 列，选 Top 3，过滤 ram 为负的标的），配合 `Threshold` 信号计算权重。打印三种方案的权重对比（取最新一天）。

4. 组装动量排名策略并运行回测（复用 COMMON 字典，`portfolio=TopNRankingOptimizer(...)`，`signal.required_indicators` 中需额外加入 Momentum 和 Ratio）。权重时间序列通过 `result.weights_df()` 提取。

5. 打印三种方案的指标对比表（累计收益率、年化波动率、最大回撤）。

6. 画三条净值曲线对比图（figsize 12×6）：

- 等权组合（灰色虚线）
- 风险平价组合（蓝色实线）
- 动量排名组合（橙色实线）
- 归一化到起点 = 100
- 标题「三种信号对比：等权 vs 风险平价 vs 动量排名」

7. 画动量排名的权重变化历史图（figsize 12×4）：

- 堆叠面积图，展示三只 ETF 权重随时间的变化
- 注意：动量为负时权重为 0（面积会出现空白）
- 标题「动量排名：涨的多买，跌的不买」

结果呈现

1. 三信号权重对比表
2. 三信号指标对比表
3. 三条净值曲线对比图
4. 动量排名权重变化历史图
5. 打印分析（根据实际数据动态描述方向，不要硬写"更大"或"更深"）：
 - 动量排名更激进——涨势好的时候集中加仓，跌的直接不买。对比风险平价的收益率变化。
 - 对比波动率和回撤的变化方向（根据数据判断是变大还是变小）。
 - 「没有完美的分法——风险平价求稳，动量排名求收益。选哪个，取决于你更怕亏还是更想赚。」

验证

执行成功的标志：

- 新增的单元格无报错运行完毕
- 动量排名权重三只 ETF 不相等，且可能有 0（动量为负的标的）
- 指标对比表包含三列，数值合理
- 净值曲线图显示 3 条线（灰色虚线 + 蓝色实线 + 橙色实线）
- 权重历史堆叠面积图中可见空白区域（对应动量为负时不持仓）

Spec: 调仓频率——多久调一次才合适？

所有命令在沙箱外运行。

上下文

本章探讨"什么时候买卖"。Q3 用了三种信号（等权、风险平价、动量排名）。本章选择风险平价（RiskParity）作为基础信号——它始终持有全部 3 只 ETF，方便我们观察止损和止盈规则的效果。Q3 每 10 个交易日调一次仓，但为什么是 10 天？调得更勤或更懒，结果会怎样？

任务

在 notebook `q4-when-to-trade.ipynb` 中创建代码，对比不同调仓频率的效果，并首次引入交易成本。

要求

1. 获取当前工作目录，以此为根目录操作。
2. 阅读以下 oxq 模块的源码，了解接口的输入、输出和参数含义：

- o `oxq.trade.PercentageFee`
- o `oxq.trade.SimBroker`（重点关注 `fee_model` 参数）
- o `oxq.rules.RebalanceFrequencyRule`
- o `oxq.portfolio.optimizers.RiskParityOptimizer`
- o `oxq.core.Engine`（重点关注 `run(rules=[...])` 参数）、`oxq.core.Strategy`

3. 创建 notebook `q4-when-to-trade.ipynb`，导入所需库并设置中文显示：

- o `from oxq.core import Engine, Strategy`
- o `from oxq.data import YFinanceDownloader, LocalMarketDataProvider`
- o `from oxq.indicators import RollingVolatility`
- o `from oxq.signals import Threshold`
- o `from oxq.portfolio.optimizers import RiskParityOptimizer`
- o `from oxq.rules import RebalanceFrequencyRule`
- o `from oxq.trade import SimBroker, PercentageFee`
- o `from oxq.universe import StaticUniverse`
- o `pandas, numpy, matplotlib.pyplot, Decimal`
- o 设置 matplotlib 中文显示支持（macOS: Arial Unicode MS / STHeiti, Windows: SimHei, `axes.unicode_minus = False`）

4. 复用 Q3 的数据和策略配置：

- o `SYMBOLS = ("510300.SS", "513100.SS", "518880.SS")`
- o `SYMBOL_NAMES = {"510300.SS": "沪深300ETF", "513100.SS": "纳指100ETF", "518880.SS": "黄金ETF"}`
- o 数据起始日期 `START = "2021-01-01"`
- o 使用 `YFinanceDownloader` 下载数据
- o 构建 `StaticUniverse`，加载数据
- o 创建 `Threshold` 信号并绑定所需指标：

```
signal = Threshold()
signal.required_indicators = {"vol": (RollingVolatility(), {"column": "close", "period": 20})}
```

- o 创建 `RiskParityOptimizer`：`portfolio = RiskParityOptimizer(volatility_col="vol")`

5. 定义 `run_backtest(frequency, fee_model, order_rules)` 辅助函数，封装策略构建和回测执行，供后续 spec 复用：

```
def run_backtest(frequency, fee_model=None, order_rules=None):
    strategy = Strategy(
        signals={"active": (signal, {"column": "close", "threshold": 0, "relationship": "gt"})},
        portfolio=portfolio,
    )
    rules = [RebalanceFrequencyRule(interval_days=frequency)]
    if order_rules:
        rules.extend(order_rules)
    engine = Engine(universe=universe, strategy=strategy, broker=SimBroker(fee_model=fee_model))
    return engine.run(rules=rules)
```

6. 定义 4 种调仓频率：`FREQUENCIES = [5, 10, 21, 63]`，分别对应约每周、两周、每月、每季度。
7. 第一轮回测——不含成本（`fee_model=None`）：

- 循环跑 4 种频率的回测
 - 打印对比表：频率、累计收益率、年化波动率、最大回撤、夏普比率、交易次数
8. 第二轮回测——含成本 (`fee_model=PercentageFee(rate=Decimal("0.001"), min_fee=Decimal("5"))`) :

- 循环跑 4 种频率的回测
 - 打印对比表：频率、累计收益率、年化波动率、最大回撤、夏普比率、交易次数、总手续费
 - 总手续费计算： `sum(f.fee for f in result.trades)`
9. 画净值曲线对比图 (`figsize 12x6`) :

- 4 条线对应 4 种频率 (含成本版本)
 - 归一化到起点 = 100
 - 标题「调仓频率对比 (含交易成本) 」
10. 打印分析 (根据实际数据动态描述方向, 不要硬写"更高"或"更低") :
- 不含成本时, 哪个频率效果最好?
 - 加上成本后, 排名有没有变化?
 - 交易次数和总手续费随频率的变化规律
 - 「每次调仓都不是免费的——佣金和手续费是实打实的钱。频率越高, 交易越多, 成本越大。」
 - 「但频率的选择不是最关键的问题。更要命的是——两次调仓之间如果某只 ETF 暴跌, 你只能干等到下一个调仓日。能不能加一个'紧急出口'? 」

结果呈现

- 1. 无成本对比表（4 行，6 列）
- 2. 含成本对比表（4 行，7 列，多一列"总手续费"）
- 3. 含成本净值曲线对比图
- 4. 分析文字

验证

执行成功的标志：

- 新增的单元格无报错运行完毕
- 无成本和含成本各有 4 行对比数据
- 含成本表多一列总手续费，手续费随频率增大而增大
- 净值曲线图显示 4 条线

Spec: 亏了就跑——止损规则

所有命令在沙箱外运行。

上下文

在 `q4-when-to-trade.ipynb` 中已有：

- RiskParity（风险平价）策略在不同调仓频率下的回测结果
- 学员已看到交易成本的影响，理解了"交易不是免费的"

当前问题：定期调仓有个硬伤——中间暴跌只能干等。能不能加个保护？

任务

在 notebook 中新建代码单元格，给策略加上止损规则，对比不同止损阈值的效果。

要求

1. 阅读以下 `oxq` 模块的源码，了解接口的输入、输出和参数含义：

- `oxq.rules.StopLossRule`（注意：`StopLossRule` 作为 `rule` 传入 `Engine.run(rules=[...])`，不再是 `Strategy` 字段）

2. 解释 `StopLossRule` 的工作原理：

- 当持有仓位时，`StopLossRule` 会以 `avg_cost × (1 - threshold)` 为止损价，提交一个 `stop SELL` 挂单给 `SimBroker`
- `SimBroker` 在后续每个 `bar` 检查：如果收盘价跌到止损价以下，自动触发卖出
- 这是一个**挂单机制**——不是立即卖，而是"预设一个底线，跌破了自动执行"
- **为什么 `Signal` 做不到这件事**：`Signal` 只看市场数据（动量、波动率），不知道你的买入价是多少。止损需要知道"我买了多少钱、现在亏了多少"——这只有 `Rule` 能做

3. 从 `spec-01` 的回测结果中，**用代码动态选出**含成本后夏普比率最高的频率作为基准（不要硬编码数字）：

```
BEST_FREQ = max(FREQUENCIES, key=lambda f: results_with_fee[f].sharpe_ratio())
```

4. 定义 4 组对比：无止损 / `threshold=0.05 / 0.10 / 0.15`

5. 全部含成本回测（使用与 `spec-01` 相同的 `fee_model`）。调用 `spec-01` 定义的 `run_backtest` 辅助函数，将止损规则作为 `order_rules` 传入：

```
run_backtest(
    frequency=BEST_FREQ,
    fee_model=PercentageFee(rate=Decimal("0.001"), min_fee=Decimal("5")),
    order_rules=[StopLossRule(threshold=T)],
)
```

在 `run_backtest` 内部，止损规则与 `RebalanceFrequencyRule` 一起传入 `Engine.run(rules=[...])`。

6. 打印对比表：止损阈值、累计收益率、年化波动率、最大回撤、夏普比率、交易次数、总手续费

7. 画净值曲线对比图（`figsize 12x6`）：

- 无止损（灰色虚线）vs 3 种止损阈值（不同颜色实线）
- 归一化到起点 = 100
- 标题「止损规则对比（含交易成本）」

8. 在无止损的净值曲线上，标注最大回撤发生的时段（用浅红色阴影区域），让学员直观看到"止损本来可以避免的坑"。

9. 打印分析（根据实际数据动态描述方向）：

- 止损对最大回撤的保护效果
- 止损触发了多少额外交易？带来了多少额外成本？
- 阈值太紧（5%）和太松（15%）的各自问题
- 「止损不是免费的——每次止损都是一次额外的卖出+之后的买回，都要付手续费。」
- 「止损保护了本金——活着才能赚钱。但阈值怎么选，是个权衡。」

结果呈现

1. StopLossRule 工作原理说明（挂单机制 + Signal vs Rule 的区别）
2. 指标对比表
3. 净值曲线对比图（含回撤标注）
4. 分析文字

验证

执行成功的标志：

- 新增的单元格无报错运行完毕
- 对比表有 4 行，数值合理
- 净值曲线图中无止损为灰色虚线，有止损为彩色实线
- 无止损曲线上有浅红色阴影标注最大回撤时段

Spec: 赚了要不要走？——止盈规则

所有命令在沙箱外运行。

上下文

在 `q4-when-to-trade.ipynb` 中已有：

- 不同调仓频率的对比结果
- 止损规则的效果：保护了本金，但有成本

当前问题：止损帮你控制了亏损。反过来——赚了一定程度，要不要主动卖掉、落袋为安？

任务

在 notebook 中新建代码单元格，给策略加上止盈规则，对比不同止盈阈值的效果。

要求

1. 阅读以下 oxq 模块的源码，了解接口的输入、输出和参数含义：

- o `oxq.rules.TakeProfitRule`（注意：`TakeProfitRule` 作为 rule 传入 `Engine.run(rules=[...])`，不再是 `Strategy` 字段）

2. 解释 `TakeProfitRule` 的工作原理：

- o 当持有仓位时，`TakeProfitRule` 会以 `avg_cost × (1 + threshold)` 为止盈价，提交一个 limit SELL 挂单给 `SimBroker`
- o `SimBroker` 在后续每个 bar 检查：如果收盘价涨到止盈价以上，自动触发卖出
- o 和止损一样是挂单机制——“预设一个目标，到了就自动锁定利润”

3. 从 spec-02 的回测结果中，用代码动态选出夏普比率最高的止损阈值，与 spec-01 选出的最优频率一起作为基准（不要硬编码数字）：

```
BEST_SL = max([sl for sl in STOP_THRESHOLDS if sl is not None],
               key=lambda sl: results_sl[sl].sharpe_ratio())
```

4. 定义 4 组对比：不止盈 / threshold=0.10 / 0.20 / 0.30

5. 全部含成本回测。调用 spec-01 定义的 `run_backtest` 辅助函数，将止损和止盈规则作为 `order_rules` 传入：

```
run_backtest(
    frequency=BEST_FREQ,
    fee_model=PercentageFee(rate=Decimal("0.001"), min_fee=Decimal("5")),
    order_rules=[StopLossRule(threshold=BEST_SL), TakeProfitRule(threshold=TP)],
)
```

在 `run_backtest` 内部，所有规则与 `RebalanceFrequencyRule` 一起传入 `Engine.run(rules=[...])`。

6. 打印对比表：止盈阈值、累计收益率、年化波动率、最大回撤、夏普比率、交易次数、总手续费

7. 画净值曲线对比图（figsize 12x6）：

- o 不止盈（灰色虚线）vs 3 种止盈阈值（不同颜色实线）
- o 归一化到起点 = 100
- o 标题「止盈规则对比（含交易成本）」

8. 打印交易记录（取止盈阈值最小的那组，展示前 10 笔），让学员看到不同 `order_type` 的区别。

9. 打印分析（根据实际数据动态描述方向）：

- o 止盈对总收益和夏普比率的影响——每个阈值分别是提高了还是降低了？
- o 与 Step 2 止损的对比：止损截断亏损（逻辑清晰），止盈截断利润（越紧伤害越大）
- o 止盈每次触发都是额外交易，手续费明显增加
- o 止盈的效果高度依赖阈值选择和市场走势，不像止损有稳定的保护逻辑
- o 「交易圈有句老话：「让利润奔跑，截断亏损」（Let profits run, cut losses short）——止损的逻辑清晰（截断亏损），止盈的逻辑模糊（你怎么知道涨到头了？）。」
- o 「加规则之前，先问自己：这条规则的逻辑站得住脚吗？数据支持吗？」

结果呈现

1. TakeProfitRule 工作原理说明
2. 指标对比表
3. 净值曲线对比图
4. 交易记录（前 10 笔）
5. 分析文字

验证

执行成功的标志：

- 新增的单元格无报错运行完毕
- 对比表有 4 行，数值合理
- 净值曲线图中不止盈为灰色虚线，有止盈为彩色实线
- 交易记录中可见 stop（止损触发）和 limit（止盈触发）类型的订单

Spec: 跟什么比？——基准与风险调整指标

所有命令在沙箱外运行。

上下文

本章评估策略的有效性。Q3-Q4 产出了多个策略变体，回测指标看起来不错。但"不错"是跟什么比？年化 15% 好不好，取决于同期市场涨了多少。

本章用四个策略作为"被检查对象"：EqualWeight（Q3 等权）、RiskParity（Q3 风险平价）、TopNRanking（Q3 动量排名）、RiskParity + 5% 止损（Q4 最终产出）。

任务

在 notebook `q5-how-to-validate.ipynb` 中创建代码，把策略和买入持有基准放在一起比较，并引入多种风险调整指标。

要求

1. 获取当前工作目录，以此为根目录操作。

2. 阅读以下 oxq 模块的源码，了解 `RunResult` 的评估接口：

- `result.total_return()` — 累计收益率
- `result.annualized_return()` — 年化收益率
- `result.annualized_volatility()` — 年化波动率
- `result.max_drawdown()` — 最大回撤
- `result.sharpe_ratio()` — 夏普比（年化收益 / 波动率）
- `result.calmar_ratio()` — 卡玛比（年化收益 / 最大回撤）
- `result.sortino_ratio()` — 索提诺比（年化收益 / 下行波动率）

3. 创建 notebook `q5-how-to-validate.ipynb`，导入所需库并设置中文显示：

- `from oxq.core import Engine, Strategy`
- `from oxq.data import YFinanceDownloader, LocalMarketDataProvider`
- `from oxq.indicators import RollingVolatility, Momentum, Ratio`
- `from oxq.portfolio.optimizers import EqualWeightOptimizer, RiskParityOptimizer, TopNRankingOptimizer`
- `from oxq.signals import Threshold`
- `from oxq.rules import RebalanceFrequencyRule, StopLossRule`
- `from oxq.trade import SimBroker, PercentageFee`
- `from oxq.universe import StaticUniverse`
- `pandas, numpy, matplotlib.pyplot, Decimal`
- 设置 matplotlib 中文显示支持（macOS: Arial Unicode MS / STHeiti, Windows: SimHei, `axes.unicode_minus = False`）

4. 复用 Q3/Q4 的数据和策略配置：

- `SYMBOLS = ("510300.SS", "513100.SS", "518880.SS")`
- `SYMBOL_NAMES = {"510300.SS": "沪深300ETF", "513100.SS": "纳指100ETF", "518880.SS": "黄金ETF"}`
- 数据起始日期 `START = "2021-01-01"`
- 使用 `YFinanceDownloader` 下载数据
- 构建 `StaticUniverse`
- `FEE_MODEL = PercentageFee(rate=Decimal("0.001"), min_fee=Decimal("5"))`

5. 先阅读 `oxq.portfolio.optimizers` 模块源码，了解

`EqualWeightOptimizer`、`RiskParityOptimizer`、`TopNRankingOptimizer` 的构造参数和 `required_indicators` 属性。

6. 定义四个 portfolio 配置常量：

- `EW_PORTFOLIO = EqualWeightOptimizer()`
- `RP_PORTFOLIO = RiskParityOptimizer(volatility_col="vol")`
- `TNR_PORTFOLIO = TopNRankingOptimizer(score_col="ram", n=3, filter_negative=True)`
- `RP_SL_PORTFOLIO = RiskParityOptimizer(volatility_col="vol")`（与 RP 相同，止损在运行时指定）

7. 定义辅助函数 `run_strategy(portfolio, indicators, freq=10, stop_loss=None)`：

- `portfolio` 是一个 optimizer 实例（如 `EqualWeightOptimizer()`）
- `indicators` 是指标列表（如 `[RollingVolatility(period=20)]`），从 `portfolio.required_indicators` 获取或手动指定
- 使用 `Threshold` 作为始终为真的信号
- 规则通过 `Engine.run(rules=[RebalanceFrequencyRule(freq), ...])` 传入

- 组装 Strategy，含交易成本
- 返回 RunResult
- 供后续 spec 复用

8. 运行四个策略：

- EqualWeight：`run_strategy(EW_PORTFOLIO, indicators=[])`
- RiskParity：`run_strategy(RP_PORTFOLIO, indicators=[RollingVolatility(period=20)])`
- TopNRanking：`run_strategy(TNR_PORTFOLIO, indicators=[RollingVolatility(period=20), Momentum(period=20), Ratio(...)])`
- RiskParity + 止损：`run_strategy(RP_SL_PORTFOLIO, indicators=[RollingVolatility(period=20)], stop_loss=0.05)`

9. 构造买入持有基准——等权买入 3 只 ETF，之后不做任何交易：

- 加载价格数据，每只 ETF 归一化到 1，等权平均
- 计算基准的累计收益、年化收益、年化波动率、最大回撤、夏普比

10. 打印策略 vs 基准对比表：

- 列：策略名、累计收益、年化收益、波动率、最大回撤、夏普比、卡玛比、索提诺比
- 最后一行是等权买入持有基准
- 额外打印超额收益 / Alpha（策略累计 - 基准累计），并呼应 Q0 提到的 Alpha 概念：「还记得 Q0 说的吗？Beta 是跟着市场赚的钱，Alpha 是比市场多赚的钱——你的技能回报。」

11. 画净值曲线对比图（figsize 12x6）：

- 4 条策略线（不同颜色实线）+ 1 条基准线（灰色虚线）
- 归一化到起点 = 100
- 标题「策略 vs 买入持有基准」
- 图例放在左上角

12. 打印分析（根据实际数据动态描述方向，不要硬写"更高"或"更低"）：

- 哪些策略跑赢了基准？哪些没有？
- 超额收益（Alpha）最大的是哪个？但卡玛比和索提诺比的排名呢？
- 「年化 15% 好不好？取决于同期等权买入持有赚了多少钱。跑赢基准的部分，才是你的'本事'——多出来的叫超额收益，也就是 Q0 说的 Alpha。」
- 「夏普比、卡玛比、索提诺比是三把不同的尺子。夏普比看总波动，卡玛比看最大回撤，索提诺比只看下行波动。你最怕什么风险，就用什么尺子。」
- 「整体指标看着还行——但这是好几年平均出来的。每一年都赚钱吗？」

结果呈现

- 1. 策略 vs 基准指标对比表 (5 行 × 8 列)
- 2. 超额收益表
- 3. 净值曲线对比图
- 4. 分析文字

验证

执行成功的标志：

- 新增的单元格无报错运行完毕
- 对比表有 5 行（4 策略 + 1 基准），8 个指标列（使用中文指标名：夏普比、卡玛比、索提诺比）
- 净值曲线图显示 5 条线（4 实线 + 1 虚线）
- 超额收益有正有负或全正

Spec: 每年都有收益吗？——分段检验

所有命令在沙箱外运行。

上下文

在 `q5-how-to-validate.ipynb` 中已有：

- 四个策略 vs 买入持有基准的整体指标对比
- 学员已了解基准、超额收益（Alpha）、卡玛比/索提诺比等概念

当前问题：整体指标是多年平均值，可能掩盖某些年份的亏损。需要逐年拆开看。

任务

在 notebook 中新建代码单元格，按年度拆解策略表现，并深挖 TopNRanking 在特定年份的异常。

要求

1. 阅读以下 oxq 接口的源码：

- `result.daily_returns()` — 返回日收益率 Series
- `result.equity_curve` — 净值序列 (list of tuples: (date_str, value))

2. 按年度拆解四个策略 + 基准：

- 用 `daily_returns()` 按年分组
- 计算每年的：年收益率（累乘）、年波动率、年夏普比、年内最大回撤
- 同时计算等权买入持有基准的逐年收益

3. 打印逐年对比表：

- 行：年份
- 列：四个策略各一列（年收益），最后一列是基准年收益
- 负收益用 * 标注

4. 画逐年收益柱状图（figsize 14x6）：

- 每年一组柱子，每个策略一种颜色 + 基准灰色
- y=0 处画一条水平线
- 标题「逐年收益对比」

5. 深挖 TopNRanking 在负收益年份：

- 用不同动量窗口 `mom_period = [5, 10, 15, 20, 25, 30]` 分别构造 `TopNRankingOptimizer(score_col="ram", n=3, filter_negative=True)` 并调用 `run_strategy(portfolio, indicators=[...])` 运行
- 提取负收益年份的年收益率
- 打印对比表：动量窗口 vs 该年收益
- 如果所有窗口都亏钱，说明不是参数问题

6. 打印分析（根据实际数据动态描述方向）：

- 哪些策略“每年都赚”？哪些有负收益年份？
- 负收益年份中，基准表现如何？
- TopNRanking 负收益年份深挖结论
- 「平均年化 10% 听着不错——但如果其中一年亏了 20%，你扛得住吗？整体指标可以掩盖很多问题，逐年拆开才能看到真实的持有体验。」
- 「TopNRanking 的负收益不是参数选错了——换遍所有窗口都亏。这是动量策略的结构性弱点：当市场反复上下波动、没有明确方向时，过去涨得好的不代表未来继续涨。动量指标有滞后性。」
- 「知道了每年的表现——但每个月呢？赚钱是稳定地赚，还是靠几次大赚？」

结果呈现

- 1. 逐年收益对比表
- 2. 逐年收益柱状图
- 3. TopNRanking 异常年份深挖表
- 4. 分析文字

验证

执行成功的标志：

- 逐年表有完整的年份覆盖，每年 5 列数据（4 策略 + 基准）
- 柱状图每年 5 根柱子
- TopNRanking 深挖表有 6 行（6 个动量窗口）
- 如果 TopNRanking 有负收益年份，深挖表展示了该年份各窗口的表现

Spec: 收益结构什么样？——分布与回撤

所有命令在沙箱外运行。

上下文

在 `q5-how-to-validate.ipynb` 中已有：

- 四个策略 vs 基准的整体指标对比
- 逐年收益拆解，发现某些策略有负收益年份
- TopNRanking 在市场反复波动时的结构性弱点

当前问题：同样赚钱的两个策略，赚法可能完全不同。需要看收益分布和回撤深度。

任务

在 notebook 中新建代码单元格，分析月度收益分布和历史回撤事件。

要求

1. 阅读以下 `oxq` 接口的源码：

- `result.monthly_returns()` — 返回月收益率 Series
- `result.drawdown_series()` — 返回回撤序列（每天距离前高跌了多少，负值）

2. 月度收益分布分析（四个策略）：

- 统计：总月数、盈利月数、亏损月数、胜率（盈利月 / 总月）
- 平均盈利月收益、平均亏损月收益、盈亏比（平均盈利 / 平均亏损绝对值）
- Profit Factor（总盈利 / 总亏损的绝对值）
- 最好月、最差月、中位数月收益
- 最长连续盈利月数、最长连续亏损月数

3. 打印月度统计对比表：

- 行：四个策略
- 列：胜率、平均盈利月、平均亏损月、盈亏比、盈亏因子、中位数月收益、最好月、最差月、最长连赢、最长连亏

4. 画月度收益分布直方图（`figsize 14x8`，`2x2` 子图）：

- 每个策略一个子图
- 用 20 个 bins
- `x=0` 处画一条红色垂直虚线
- 子图标题含策略名和盈亏因子

5. 回撤深度分析（四个策略）：

- 用 `drawdown_series()` 获取回撤序列
- 识别每次回撤事件（回撤从 0 跌下去，到恢复到 0）：
 - 开始日期、最低点日期、恢复日期（如果尚未恢复则标“未恢复”）
 - 深度（最低点的值）
 - 下跌天数（开始到最低点）
 - 恢复天数（最低点到恢复，未恢复则标“—”）
 - 总持续天数
- 按深度排序，打印前 5 大回撤事件

6. 打印回撤汇总：最大回撤、平均回撤深度、平均恢复时间、最长恢复时间

7. 画水下曲线（`figsize 14x10`，4 个子图垂直排列，共享 `x` 轴）：

- 每个策略一条水下曲线
- 用 `fill_between` 从 0 往下填充为浅蓝色
- 子图标题含策略名和最大回撤值
- `x` 轴只在最底部子图显示

8. 拆解 TopNRanking 2021 年月度收益（用数据说明胜率陷阱）：

- 提取 TopNRanking 2021 年的月度收益序列
- 计算 2021 年的：胜率、平均盈利月收益、平均亏损月收益、盈亏比、累计收益
- 打印 2021 年逐月明细
- 打印四个策略 2021 年的月度对比表（胜率、平均盈利月、平均亏损月、盈亏比、年收益）
- 对比说明：TopNRanking 2021 年的胜率未必低，但盈亏比远低于其他策略——赚的月份赚得少、亏的月份亏得狠，导致全年亏钱

9. 打印分析（根据实际数据动态描述方向）：

- 胜率陷阱：零基础学员天然追求高胜率，但胜率本身意义不大——真正决定收益的是盈亏比

- 用 2021 年四个策略的月度对比数据说明：同样的胜率，完全不同的结局——盈亏比才决定最终收益
- 盈亏因子（Profit Factor）的含义：大于 1 说明赚的比亏的多，大于 2 算优秀
- 最大回撤 vs 恢复时间对比
- 「很多优秀的策略胜率并不高——它们的秘诀是'小亏大赚'：用止损控制每次亏损的幅度（小亏），赶上趋势时不封顶地赚（大赚）。追求高胜率，往往是捡了芝麻丢了西瓜。」
- 「水下曲线告诉你一个残酷的事实：你有多少时间是在'还没回本'的状态中度过的。回撤不只是一个数字——它是一段煎熬。」
- 「评估够全面了吗？其实还有一个隐藏问题：你用的参数（动量窗口 20 天、波动率窗口 20 天）是不是恰好选对了？」

结果呈现

1. 月度统计对比表 (4 行 × 8 列)
2. 月度收益分布直方图 (2×2)
3. TopNRanking 2021 年逐月明细 + 四策略 2021 年月度对比表
4. 前 5 大回撤事件表 (每个策略)
5. 回撤汇总表
6. 水下曲线图 (4 个子图)
7. 分析文字

验证

执行成功的标志：

- 月度统计表有 4 行，胜率在 0-100% 之间
- 直方图 4 个子图各有一条红色 $x=0$ 虚线
- 每个策略列出了前 5 大回撤事件（或全部事件如不足 5 个）
- 水下曲线图有 4 个子图，曲线始终 ≤ 0

Spec: 参数动一动，崩不崩？——参数敏感性

所有命令在沙箱外运行。

上下文

在 `q5-how-to-validate.ipynb` 中已有：

- 四个策略的整体指标、逐年收益、月度分布、回撤深度分析
- 学员已建立了多维度评估的认知

当前问题：所有分析都基于特定参数（`period=20, frequency=10, threshold=0.05`）。如果参数稍微变一下，结论还成立吗？

任务

在 notebook 中新建代码单元格，扫描关键参数的敏感性，对比不同策略的稳健程度。

要求

1. RiskParity 波动率窗口敏感性：

- 扫描 `vol_period = [10, 15, 20, 25, 30, 40]`
- 对每组参数构造 `RiskParityOptimizer(volatility_col="vol")` 并调用 `run_strategy(portfolio, indicators=[RollingVolatility(period=vol_period)], freq=10)`
- 打印对比表（6行×5列）：窗口、累计收益、夏普比、卡玛比、最大回撤

2. TopNRanking 动量窗口敏感性：

- 扫描 `mom_period = [10, 15, 20, 25, 30, 40]`（波动率窗口固定 20）
- 对每组参数构造 `TopNRankingOptimizer(score_col="ram", n=3, filter_negative=True)` 并调用 `run_strategy(portfolio, indicators=[...], freq=10)`
- 打印对比表（6行×5列）：窗口、累计收益、夏普比、卡玛比、最大回撤

3. 止损阈值敏感性：

- 标题注明「基于 RiskParity 策略，调仓频率 10 天」
- 扫描 `thresholds = [0.02, 0.03, 0.05, 0.07, 0.10, 0.15, 0.20]` + 无止损
- 对每组参数调用 `run_strategy(RiskParityOptimizer(volatility_col="vol"), indicators=[...], freq=10, stop_loss=threshold)`
- 打印对比表（8行×5列）：止损阈值、累计收益、夏普比、最大回撤、交易次数

4. 计算每组的夏普比极差（最大夏普比 - 最小夏普比），打印对比：

- RiskParity 夏普比极差
- TopNRanking 夏普比极差
- 标注哪个是“高原型”（极差小），哪个是“山峰型”（极差大）

5. 画参数敏感性折线图（figsize 15x5，1x3 子图）：

- 子图 1：RiskParity `vol_period`（x 轴）vs 夏普比（y 轴），标题「RiskParity 波动率窗口」
- 子图 2：TopNRanking `mom_period`（x 轴）vs 夏普比（y 轴），标题「TopNRanking 动量窗口」
- 子图 3：止损阈值（x 轴）vs 夏普比（y 轴），无止损用虚线标注，标题「RiskParity 止损阈值」
- 三个子图的 y 轴范围统一，方便对比波动幅度

6. 打印分析（根据实际数据动态描述方向）：

- RiskParity 的夏普比极差 vs TopNRanking 的夏普比极差
- 哪个是“高原型”，哪个是“山峰型”
- 止损阈值是否存在一个“稳定区间”
- 「想象你在山上找营地。高原型策略像一片平坦的高原——站在哪里都差不多高。山峰型策略像一座尖峰——只有山尖上最高，稍微偏一步就掉下去。你选哪个？」
- 「好策略不依赖某个‘魔法参数’。如果换一个参数值结果就崩了，说明你可能只是运气好选对了那个值。」

结果呈现

- 1. RiskParity 波动率窗口敏感性表 (6 行 × 5 列)
- 2. TopNRanking 动量窗口敏感性表 (6 行 × 5 列)
- 3. 止损阈值敏感性表 (8 行 × 5 列, 含交易次数)
- 4. 夏普比极差对比
- 5. 参数敏感性折线图 (1×3)
- 6. 分析文字

验证

执行成功的标志：

- 三组表各有正确的行列数
- 折线图 3 个子图，y 轴范围一致
- 夏普比极差有明确数值，可判断高原型 vs 山峰型
- RiskParity 极差明显小于 TopNRanking 极差

Spec: 能不能找到最赚钱的参数？——参数优化与样本内/样本外

所有命令在沙箱外运行。

上下文

本章探讨参数优化和过拟合。Q5 的参数敏感性分析使用了默认参数（动量窗口 20、波动率窗口 20、调仓频率 10），看到了 plateau vs peak 的差异。但学员从未尝试过"从一堆参数中找出最好的"——现在要迈出这一步。

承接 Q5 的叙事：TopNRanking 在 2021 年差市场的默认参数，在 2017-2020 好市场上表现不错。如果我们在好市场上优化参数，找到"最赚钱"的组合，未来遇到差市场会怎么样？

任务

在 notebook `q6-avoid-overfitting.ipynb` 中创建代码，用 `oxq.optimize` 实现参数网格搜索，并用样本内/样本外方法检验过拟合。

要求

1. 获取当前工作目录，以此为根目录操作。

2. 阅读以下 oxq 模块的源码，了解接口的输入、输出和参数含义：

- `oxq.optimize.ParameterSet` — 参数空间定义 (`add()`, `add_constraint()`, `grid()`)
- `oxq.optimize.GridSearch` — 网格搜索器 (`run()` → `SearchResult`)
- `SearchResult` 的接口： `best`, `top_n()`, `to_dataframe()`
- `TrialResult` 的属性： `params`, `metric_value`, `run_result`

3. 创建 notebook `q6-avoid-overfitting.ipynb`，导入所需库并设置中文显示：

- `from oxq.core import Engine, Strategy`
- `from oxq.data import YFinanceDownloader, LocalMarketDataProvider`
- `from oxq.indicators import RollingVolatility, Momentum, Ratio`
- `from oxq.signals import Threshold`
- `from oxq.portfolio.optimizers import TopNRankingOptimizer`
- `from oxq.rules import RebalanceFrequencyRule, StopLossRule`
- `from oxq.trade import SimBroker, PercentageFee`
- `from oxq.universe import StaticUniverse`
- `from oxq.optimize import ParameterSet, GridSearch`
- `from oxq.optimize.search import _apply_params, _apply_rule_params`
- `pandas, numpy, matplotlib.pyplot, Decimal`
- 设置 matplotlib 中文显示支持 (macOS: Arial Unicode MS / STHeiti, Windows: SimHei, `axes.unicode_minus = False`)

4. 复用之前的数据和配置：

- `SYMBOLS = ("510300.SS", "513100.SS", "518880.SS")`
- 数据起始日期 `2017-01-01`
- `FEE_MODEL = PercentageFee(rate=Decimal("0.001"), min_fee=Decimal("5"))`

5. 构造基础策略 (`TopNRanking`)，阅读 `Strategy` 源码了解构造方式：

- 信号： `Threshold` (`always-true` 信号)，设置 `signal.required_indicators` 包含 `RollingVolatility` (`period=20`)、`Momentum` (`period=20`)、`Ratio` (`mom/vol`)
- 组合优化器： `TopNRankingOptimizer(score_col="ram", n=3, filter_negative=True)`
- Strategy 构造： `Strategy(signals={"active": (signal, {"column": "close", "threshold": 0, "relationship": "gt"})}, portfolio=TopNRankingOptimizer(...))`
- 规则传入 `Engine.run()`： `rules=[RebalanceFrequencyRule(interval_days=10)]`

6. 数据切割——样本内/样本外 (时间顺序正确，无前视问题)：

- 找到所有标的的共同交易日
- 样本内：2017-01-01 ~ 2020-12-31 (好市场，用来优化参数)
- 样本外：2021-01-01 ~ 2023-01-31 (差市场，用来验证)
- 打印两段的起止日期和天数

7. 用 `ParameterSet` 定义参数空间：

- 动量和波动率窗口： `PERIODS = [10, 15, 20, 25, 30]`
- 调仓频率： `FREQS = [5, 10, 15, 21]`
- 止损阈值： `SL_THRESHOLDS = [0.03, 0.05, 0.07, 0.10, 0.15, 0.20]`
- 无止损组： `mom.period × vol.period × RebalanceFrequencyRule.interval_days`
- 含止损组：同上 + `StopLossRule.threshold`

- 约束：调仓频率不应超过信号窗口（`RebalanceFrequencyRule.interval_days <= mom.period` , `RebalanceFrequencyRule.interval_days <= vol.period`）
- 用 `add_constraint(expr)` 添加约束，格式：`"component.param op component.param"`
- 打印约束后的有效组合数（用 `len(paramset.grid())`）

8. 网格搜索：

- `GridSearch.run()` 不支持直接传入 rules，需要编写 `grid_search_with_rules()` 包装函数
- 包装函数使用 `_apply_params` 和 `_apply_rule_params`（从 `oxq.optimize.search` 导入）将参数应用到 `strategy` 和 `rules` 上
- `broker_factory` 使用 `lambda: SimBroker(fee_model=FEE_MODEL)`
- 分别对无止损组和含止损组，在样本内和样本外各跑一次
- 合并结果

9. 分析与展示：

- 按样本内夏普排名，打印 Top 20 及其样本外表现
- 打印 Top 20 的样本内/样本外夏普统计（均值、范围）
- 计算全局样本内/样本外相关系数
- 计算全部组合的夏普衰减分布（均值、中位数）
- 计算样本外比样本内差的比例

10. 画两张图：

- 图 1（`figsize 10×8`）：样本内夏普 vs 样本外夏普散点图
 - 每个点是一个参数组合
 - 画 $y=x$ 对角线（虚线）——对角线以下说明样本外不如样本内
 - 标注 Top 20（红色大点）
 - 标题「N 个参数组合：样本内 vs 样本外夏普比」
- 图 2（`figsize 12×6`）：Top 20 的样本内/样本外夏普对比柱状图
 - 每组两根柱子（样本内 / 样本外）
 - 标题「样本内 Top 20 的“样本外真相”」

11. 打印分析（根据实际数据动态描述）：

- 样本内 Top 20 的平均夏普，以及样本外的平均夏普
- 散点图中点集的分布特征（多数在对角线以上还是以下）
- 总结：参数优化可以帮你找到更好的参数，但如果只看优化时段的结果就做决定，可能只是“记住了”这段数据的特征，而不是发现了真规律。这种现象叫过拟合。
- 过渡：我们需要更科学的验证方法——只验证一次够吗？市场在变，参数也该跟着变吧？

验证

执行成功的标志：

- 新增的单元格无报错运行完毕
- 参数空间包含无止损组和含止损组，约束后总组合数约 490
- Top 20 表有 20 行，包含样本内夏普和样本外夏普
- 散点图显示所有组合的样本内 vs 样本外夏普，Top 20 用红色标注
- 柱状图显示 Top 20 的样本内/样本外对比

Spec: 市场在变，参数也该跟着变吧？——Walk-Forward 分析

所有命令在沙箱外运行。

上下文

在 `q6-avoid-overfitting.ipynb` 中已有：

- 490 个参数组合的网格搜索结果
- 样本内/样本外对比，发现严重的过拟合现象
- 学员已理解"把数据切一刀"的验证方法

当前问题：数据只切了一次，只验证了一次。市场在变，2023 年选的参数 2025 年还适用吗？能不能模拟"定期重新优化"的过程？

任务

在 notebook 中新建代码单元格，用 `oxq.optimize.WalkForward` 实现 Walk-forward 分析，对比 Rolling 和 Anchored 两种方式。

要求

1. 阅读以下 oxq 模块的源码，了解接口的输入、输出和参数含义：

- o `oxq.optimize.WalkForward` — Walk-forward 分析器
 - 构造参数：`paramset` , `train_period` , `test_period` , `step` , `anchored`
 - `train_period / test_period` 格式：`"2Y"` , `"6M"` , `"126D"` 等
 - `run()` → `WalkForwardResult`
- o `WalkForwardResult` 的接口：
 - `windows` — 每折的 `WindowResult` 列表
 - `deterioration()` — 返回样本内 → 样本外衰减比例（负值 = 过拟合）
 - `to_dataframe()` — 含训练/验证时段、参数、指标的 `DataFrame`
 - `oos_sharpe_ratio()` , `oos_total_return()` , `oos_max_drawdown()`
- o `WindowResult` 的属性：`train_start` , `train_end` , `test_start` , `test_end` , `best_params` , `in_sample_metric` , `oos_result`
- o 导入：`from oxq.optimize import WalkForward`

2. 复用 Step 1 的参数范围和约束（无止损组）：

- o `PERIODS = [10, 15, 20, 25, 30]`
- o `FREQS = [5, 10, 15, 21]`
- o 约束：`RebalanceFrequencyRule.interval_days <= mom.period` , `RebalanceFrequencyRule.interval_days <= vol.period`
- o 约束后 70 组合

3. Rolling Walk-Forward：

- o 训练窗口 `"2Y"` , 测试窗口 `"6M"` , `anchored=False`
- o 数据范围：`DATA_START` 到当前日期
- o 用 `rolling_result.to_dataframe()` 查看每折详情
- o 用 `rolling_result.deterioration()` 查看衰减比例

4. Anchored Walk-Forward：

- o 同样参数, `anchored=True`
- o 训练窗口从起点不断扩大

5. 打印每折的详情 + 汇总：

- o 用 `to_dataframe()` 打印每折：训练时段、验证时段、最优参数、样本内/样本外夏普
- o 汇总：`oos_sharpe_ratio()` , `deterioration()`
- o 从 `windows` 中提取参数列表，判断参数稳定性

6. Rolling vs Anchored 对比：

- o 样本外整体夏普：`oos_sharpe_ratio()`
- o 衰减比例：`deterioration()`
- o 参数稳定性（从 `windows` 的 `best_params` 提取对比）

7. 画 Walk-forward 结果图（figsize 14×6）：

- o 上半部分：每折的样本外夏普（Rolling 和 Anchored 各一条线）
- o 下半部分：参数稳定性线图——从 `windows` 的 `best_params` 提取 `mom.period` 和 `interval_days`，展示每折选择的参数变化趋势
- o 标题「Walk-Forward 分析：Rolling vs Anchored」
- o 注意：monospace 字体区域（如 `family="monospace"` 的文本）只使用 ASCII 字符，不使用中文

8. 打印分析（根据实际数据动态描述）：

- o Anchored 的参数在各折之间是否稳定——参数稳定是好信号

- Rolling 的参数在各折之间是否变化明显——如果最优参数总在变，怎么知道现在的参数下个月还是最优？
- `deterioration()` 的量化对比：Rolling 衰减 vs Anchored 衰减
- Walk-forward 的核心价值：不是找"永远最优"的参数，而是检验优化方法在历史上是否靠谱
- 过渡：Walk-forward 只用了一种切法——如果数据切的方式不同，结论还一致吗？

验证

执行成功的标志：

- 新增的单元格无报错运行完毕
- Rolling 和 Anchored 各产出多折结果（折数取决于数据长度和窗口设置）
- 每折包含训练时段、验证时段、最优参数、样本内/样本外夏普
- 对比图显示两种方式的样本外夏普趋势和参数稳定性
- `deterioration()` 返回有效数值

Spec: 一条路径够可靠吗？——交叉验证

所有命令在沙箱外运行。

上下文

在 `q6-avoid-overfitting.ipynb` 中已有：

- 参数优化 + 样本内/样本外对比
- Walk-forward 分析 (Rolling + Anchored)
- 学员已理解参数稳定性的重要性

当前问题：Walk-forward 只用了一种切法。如果换一种切法，结论还一样吗？一次验证和多次验证，可信度差多少？

任务

在 notebook 中新建代码单元格，用 `oxq.optimize.TimeSeriesCV` 实现时间序列交叉验证。

要求

1. 阅读以下 oxq 模块的源码，了解接口的输入、输出和参数含义：

- o `oxq.optimize.TimeSeriesCV` — 时间序列交叉验证
 - 构造参数：`n_splits`（折数），`embargo_days`（隔离天数），`expanding`（扩展/固定窗口）
 - `cross_validate()` → `CVResult`（传入 `paramset` 时自动在每折训练集上做 GridSearch）
 - `split()` → `list[CVSplit]`（仅生成切分，不跑回测）
- o `CVResult` 的接口：
 - `splits` — 每折的 `CVSplitResult` 列表
 - `mean_oos_metric()` — 样本外指标均值
 - `std_oos_metric()` — 样本外指标标准差
 - `to_dataframe()` — 含时段、参数、指标的 `DataFrame`
- o `CVSplitResult` 的属性：`split`（含 train/test 日期），`best_params`，`in_sample_metric`，`oos_result`
- o 导入：`from oxq.optimize import TimeSeriesCV`

2. 复用 Step 2 的参数空间和约束（无止损组，70 组合）。

3. TopNRankingOptimizer 交叉验证：

- o `n_splits=5`，`expanding=True`
- o 数据范围：`DATA_START` 到当前日期
- o 传入 `paramset` 让 `cross_validate()` 自动做每折的 GridSearch

4. 分析结果：

- o 用 `cv_result.to_dataframe()` 打印每折详情
- o `cv_result.mean_oos_metric()` — 样本外夏普均值
- o `cv_result.std_oos_metric()` — 样本外夏普标准差
- o 从 `cv_result.splits` 提取每折的 `best_params`，判断参数一致性

5. 画交叉验证结果图（figsize 12×6）：

- o 柱状图：每折的样本内夏普和样本外夏普
- o 画样本外均值水平线
- o 标题「5 折交叉验证：每折的样本内 vs 样本外夏普比」

6. 打印分析（根据实际数据动态描述）：

- o 5 折中有几折选了相同的参数——参数一致性
- o 样本外夏普的范围和标准差——策略对市场环境的敏感度
- o 交叉验证的价值：不是给你“最优参数”，而是告诉你“参数选择有多可靠”
- o 过渡：到这里，我们检验了参数选择的多种方式。但还有一种过拟合更隐蔽——不是参数选错了，而是规则太多了。

验证

执行成功的标志：

- 新增的单元格无报错运行完毕
- 5 折结果表有 5 行，包含训练/验证时段、最优参数、样本内/样本外夏普
- 汇总包含 `mean_oos_metric() ± std_oos_metric()`
- 柱状图显示 5 折的样本内/样本外夏普对比，含均值水平线

Spec: 规则越多回测越好——是好事吗？——规则负担

所有命令在沙箱外运行。

上下文

在 `q6-avoid-overfitting.ipynb` 中已有：

- 参数优化 + 样本内/样本外
- Walk-forward (Rolling + Anchored)
- 交叉验证 (5 折 Time-Series Split)

当前问题：前三步都在检验"参数选得对不对"。但还有一种过拟合来自"规则太多"——每加一条规则，就多了一个可以调节的旋钮。旋钮越多，越容易在样本内"调出"好看的结果。

任务

在 notebook 中新建代码单元格，通过两个堆叠实验展示规则负担。

要求

1. 阅读以下 oxq 模块的源码，了解可用的交易规则及其参数：

- `oxq.rules.StopLossRule` — 止损规则 (`threshold` 参数)
- `oxq.rules.TakeProfitRule` — 止盈规则 (`threshold` 参数)
- `oxq.rules.TrailingStopRule` — 移动止损规则 (`trail_pct` 参数)
- `oxq.rules.MaxDrawdownRule` — 最大回撤风控 (`max_drawdown` 参数)
- `oxq.core.Engine` — 回测引擎 (`Engine().run(strategy, market=..., broker=..., start=..., end=..., rules=[...])` 注意：构造函数无参数，rules 传入 `run()`)
- `oxq.rules.RebalanceFrequencyRule` — 调仓频率规则 (`interval_days` 参数)
- 确认 `ParameterSet.add_constraint()` 的约束表达式格式

2. 4A 规则堆叠实验：

- 基础策略：TopNRankingOptimizer，固定参数 `mom=20, vol=20, interval_days=10`
- 规则传入 `Engine().run(rules=[...])`，使用 `grid_search_with_rules()` 包装函数优化
- `run_layer` 函数签名简化为 (`layer_name, rules, paramset, description`)
- 逐层添加规则，每层在样本内优化该层的参数：
 - Layer 0：基础策略 (固定参数，无额外规则)
 - Layer 1：+ 优化调仓频率 (`interval_days = FREQS`，约束 `interval_days <= mom.period`)
 - Layer 2：+ 优化止损 (`sl = SL_THRESHOLDS`)
 - Layer 3：+ 优化止盈 (`tp = TP_THRESHOLDS`，约束 `sl < tp`)
 - Layer 4：+ 优化移动止损 (`trail = TRAIL_PCTS`，约束 `trail >= sl`)
 - Layer 5：+ 最大回撤风控 (`max_dd = MAX_DD_VALS`，约束 `max_dd >= sl`)
- 每层在样本内选最优 → 在样本外验证
- 约束过滤不合理组合 (止损 > 止盈、移动止损 < 固定止损等)
- Layer 0 没有参数可优化，直接用 `Engine().run(rules=[RebalanceFrequencyRule(interval_days=10)])` 跑回测
- 打印每层的样本内/样本外夏普、收益、衰减

3. 4B 指标堆叠实验：

- 从 1 个参数开始，逐步增加可优化的参数维度：
 - A：只优化动量窗口 (1 个自由度，PERIODS)
 - B：+ 波动率窗口 (2 个自由度)
 - C：+ 调仓频率 (3 个自由度，约束 `interval_days <= mom, interval_days <= vol`)
 - D：+ 止损阈值 (4 个自由度，约束同 C)
 - E：+ 止盈 + 移动止损 (6 个自由度，粗网格 [10,20,30]，约束 `interval_days <= mom, interval_days <= vol, sl=sl`)
- 每层在样本内优化所有参数 → 样本外验证
- 约束过滤不合理组合，与 4A 一致
- 打印每层的自由度、组合数、样本内/样本外夏普、衰减

4. 打印两个汇总表：

- 4A：层名、样本内夏普、样本外夏普、衰减、样本内收益、样本外收益、自由度
- 4B：配置名、自由度、样本内夏普、样本外夏普、衰减、样本内收益、样本外收益

5. 画两张图：

- 图 1 (`figsize 12×6`)：4A 规则堆叠——样本内夏普和样本外夏普随层数的变化 (双折线)
 - x 轴：Layer 0-5
 - 两条线：样本内 (蓝色上升) vs 样本外 (红色先升后平/降)
 - 标题「规则堆叠：样本内 vs 样本外」
- 图 2 (`figsize 12×6`)：4B 指标堆叠——同样画样本内/样本外双折线

- x 轴：自由度 1-6
- 标题「自由度越多 ≠ 越好」

6. 打印分析（根据实际数据动态描述）：

- 4A 结果：样本内夏普随规则增加的变化趋势，以及样本外的对应变化
- 规则负担的含义：每条新规则增加一个可调旋钮，优化器多了一个维度可以"调出"好看的结果
- 4B 结果：自由度增加后，样本内提升多少，样本外提升多少——多出来的"提升"是自欺欺人
- 引用 Brian Peterson 的观点：Rule burden is a form of overfitting
- 总结：能用 2 个参数解决的问题，不要用 6 个。简单就是美。

验证

执行成功的标志：

- 新增的单元格无报错运行完毕
- 4A 汇总表有 6 行（Layer 0-5），包含样本内/样本外夏普和衰减
- 4B 汇总表有 5 行（A-E），包含自由度、组合数、样本内/样本外夏普和衰减
- 两张图各显示两条折线（样本内和样本外）
- 4A 中样本内夏普随层数递增（规则越多回测越好看）

Spec: 35% 是多少股？——目标权重转具体订单

所有命令在沙箱外运行。

上下文

学员跑完 Q6，策略通过了所有检验。今天是调仓日，风险平价信号给出目标权重：沪深300ETF 35% / 纳指100ETF 25% / 黄金ETF 40%。打开券商 APP——只能输入股数和价格。35% 是多少股？

任务

在 notebook `q7-execution.ipynb` 中创建代码，用 `generate_orders()` 把目标权重翻译成具体订单，并对比 A 股（`lot_size=100`）和美股（`lot_size=1`）的取整偏差。

要求

1. 获取当前工作目录，以此为根目录操作。

2. 阅读以下 oxq 模块的源码，了解接口的输入、输出和参数含义：

- `oxq.trade.order_generator` — `generate_orders()` 函数和 `PlannedOrder` 数据类型
 - `generate_orders(target_weights, positions, prices, total_capital, lot_size) -- list[PlannedOrder]`
 - `PlannedOrder` 属性：`order` (`Order` 对象)、`current_shares`、`target_shares`、`current_weight`、`target_weight`、`estimated_amount`
- `oxq.core.types` — `Position` 数据类型 (`symbol` , `shares` , `avg_cost`)

3. 创建 notebook `q7-execution.ipynb`，导入所需库并设置中文显示：

- `from oxq.trade import generate_orders, PlannedOrder`
- `from oxq.core.types import Position`
- `from decimal import Decimal`
- `pandas, numpy, matplotlib.pyplot`
- 设置 `matplotlib` 中文显示支持 (`macOS`: Arial Unicode MS / `STHeiti`, `Windows`: `SimHei`, `axes.unicode_minus = False`)

4. 定义常量：

- A 股标的：`SYMBOLS = ("510300.SS", "513100.SS", "518880.SS")`
- 中文名映射：`SYMBOL_NAMES = {"510300.SS": "沪深300ETF", "513100.SS": "纳指100ETF", "518880.SS": "黄金ETF"}`
- 目标权重：沪深300ETF 35%、纳指100ETF 25%、黄金ETF 40%
- 模拟价格（近期收盘价）：510300.SS ¥4.27、513100.SS ¥1.72、518880.SS ¥8.04

5. 第一步——从空仓开始生成 A 股订单：

- 10 万人民币，`lot_size=100`，空仓 (`positions={}`)
- 打印订单表格：标的（中文名）、方向、目标股数、预估金额、目标权重、实际权重、偏差
- 计算并打印投入金额和剩余现金（残余占比）

6. 第二步——A 股 vs 美股取整偏差对比：

- 美股标的和价格：SPY \$593.12、QQQ \$512.30、GLD \$298.50
- 美股 10 万美元，`lot_size=1`
- 对比表格：目标权重 -- A 股（股数/实际权重/偏差）vs 美股（股数/实际权重/偏差）
- 打印两个市场的现金残余

7. 第三步——资金量对取整偏差的影响：

- 统一资金档位：1 万、5 万、10 万、50 万、100 万
- 分别计算 A 股 (`lot_size=100`) 和美股 (`lot_size=1`) 在各资金量下的最大权重偏差和现金残余
- 打印两张表格（A 股/美股各一张）

8. 画一张图 (`figsize 14×5`，两个子图并排)：

- 左图：A 股 vs 美股的最大权重偏差随资金量变化（分组柱状图）
- 右图：A 股 vs 美股的现金残余占比随资金量变化（分组柱状图）
- A 股红色、美股蓝色

9. 打印分析：

- 回测的权重精确到小数点后 N 位，现实只能买整数股
- A 股 100 股整手约束让偏差远大于美股的 1 股单位

- 资金量越小，偏差越大
- 过渡：订单有了，但按什么价格下单？

验证

执行成功的标志：

- 单元格无报错运行完毕
- A 股订单表格显示 3 只 ETF 的目标股数、实际权重与偏差
- A 股 vs 美股对比表展示取整偏差差异
- 柱状图直观展示资金量和市场对偏差的影响
- 剩余现金（现金残余）有明确数值

Spec: 价格变了怎么办？——成交价压力测试

所有命令在沙箱外运行。

上下文

在 `q7-execution.ipynb` 中已有：

- Step 1 的订单生成实验（目标权重转具体订单，A 股 vs 美股取整偏差）

当前问题：Step 1 生成了订单"买沪深300ETF 8100 股，按 ¥4.27"。但这是昨天收盘价，现在下单价格已经变了。按哪个价格买？差异有多大影响？

任务

在 notebook 中新建代码单元格，用 `FillPriceMode` 的五种模式做压力测试，再用四层成本层叠展示佣金、印花税和滑点的持续侵蚀。

要求

1. 阅读以下 oxq 模块的源码，了解接口的输入、输出和参数含义：

- o `oxq.trade.sim_broker` — `SimBroker` 和 `FillPriceMode` 枚举
 - `FillPriceMode` : `CLOSE` (收盘价)、`NEXT_OPEN` (次日开盘价)、`NEXT_HIGH` (次日最高价)、`NEXT_LOW` (次日最低价)、`MID` (最高与最低均价)
 - `SimBroker(fee_model=..., slippage_model=..., fill_price_mode=...)`
- o `oxq.trade.fees` — `PercentageFee(rate, min_fee)`
- o `oxq.trade.slippage` — `PercentageSlippage(rate)`
- o `oxq.core.Engine` — `Engine().run(strategy, market=..., broker=..., start=..., end=...)`
- o `oxq.data` — `YFinanceDownloader`, `LocalMarketDataProvider`
- o 策略相关：`make_strategy` 辅助函数、`Threshold` (信号)、`RiskParityOptimizer` (组合优化器)、`RebalanceFrequencyRule`、`StopLossRule`
 - `from oxq.signals import Threshold`
 - `from oxq.portfolio.optimizers import RiskParityOptimizer`
 - `from oxq.rules import RebalanceFrequencyRule, StopLossRule`

2. 下载数据并加载：

- o 标的：`510300.SS`、`513100.SS`、`518880.SS`
- o 起始日期：`2021-01-01`，结束：当前日期
- o 用 `YFinanceDownloader` 下载，下载失败时用 `try/except` 捕获，打印提示并使用本地缓存
- o 用 `LocalMarketDataProvider` 加载

3. 构造策略和规则 (风险平价 + 止损，与 Q4 相同逻辑)：

- o 用 `make_strategy(name)` 辅助函数构造策略：创建 `Threshold` 信号，配置 `required_indicators` (`RollingVolatility`)，使用 `RiskParityOptimizer(volatility_col="vol")` 作为 portfolio optimizer
- o 用 `make_rules()` 辅助函数构造规则：`[RebalanceFrequencyRule(interval_days=21), StopLossRule(threshold=0.10)]`
- o 所有 `Engine.run()` 调用需传入 `rules=make_rules()` 参数

4. 五种成交价模式压力测试：

- o 佣金模型：`PercentageFee(rate=Decimal("0.0006"), min_fee=Decimal("5"))` (A 股佣金万一 + 印花税折算，合计约万六)
- o 对五种 `FillPriceMode` 分别创建 `SimBroker`，跑完整回测
- o 打印对比表：成交价模式、累计收益率、年化收益率、最大回撤、夏普比率、交易次数

5. 画净值曲线图 (figsize 12×8，上下两个子图，高度比 3:1)：

- o 上图：五条净值曲线 (起点归一化到 100)，颜色区分，含图例
- o 下图：与收盘价基准的差距。画四条差距折线 (`NEXT_OPEN`、`NEXT_HIGH`、`NEXT_LOW`、`MID` 各自减去 `CLOSE`)，颜色和上图对应。额外在 `NEXT_HIGH` 和 `NEXT_LOW` 之间用浅橙色 `fill_between` 填充，标注"执行不确定性区间"
- o 用 `layout='constrained'` 而非 `plt.tight_layout()`
- o 打印分析：收盘价收益、各模式与理想的差距、最好和最差之间的差距

6. 四层成本层叠分析 (次日开盘价模式)：

- o 第一层：无成本 — `SimBroker(fill_price_mode=FillPriceMode.NEXT_OPEN)`
- o 第二层：佣金 — `PercentageFee(rate=Decimal("0.0001"), min_fee=Decimal("5"))` (万一佣金)
- o 第三层：佣金 + 印花税 — `PercentageFee(rate=Decimal("0.0006"), min_fee=Decimal("5"))` (万一佣金 + 卖出印花税 0.05% 折算为双边约万五，合计约万六)
- o 第四层：佣金 + 印花税 + 滑点 — 上述费率 + `PercentageSlippage(rate=Decimal("0.001"))`

- 打印对比表：成本层级、累计收益率、年化收益率、夏普比率
- 打印说明：印花税实际只在卖出时收取 0.05%，这里用 PercentageFee 近似折算为双边费率，实际成本略有差异

7. 画成本层叠图 (figsize 12×8，上下两个子图，高度比 3:1)：

- 先将四条权益曲线归一化到起点=100 (`equity / equity.iloc[0] * 100`)，对齐到公共交易日
- 上图：四条归一化净值曲线
- 下图：成本侵蚀折线——第二/三/四层的归一化净值分别减去"无成本"的归一化净值。三条差距线应始终 ≤ 0 (成本只会减少收益)，如果出现正值说明计算有误
- 用 `layout='constrained'`
- 打印成本侵蚀分解：佣金贡献、印花税贡献、滑点贡献
- 过渡：这些都是估算，真实执行到底差多少？

验证

执行成功的标志：

- 单元格无报错运行完毕
- 五种成交价模式的对比表有 5 行，含收益和夏普
- 净值曲线图展示五条线和执行不确定性区间填充
- 成本层叠表有 4 行，成本逐层递增时收益逐层递减
- 成本层叠图展示四条线和成本侵蚀曲线

Spec: 回测 vs 实盘，差距有多大？——Alpaca 模拟交易

所有命令在沙箱外运行。

上下文

在 `q7-execution.ipynb` 中已有：

- Step 1 的订单生成实验（A 股 ETF 取整偏差 + 美股对比）
- Step 2 的成交价压力测试和成本层叠（A 股 ETF，含印花税）

前两步用 A 股 ETF 和 SimBroker 暴露了取整偏差、滑点和成本，但都是"估算"。现在切换为美股 ETF（SPY、QQQ、GLD），接入 Alpaca 模拟交易平台，让策略真正跑一遍——open-xquant 内置了 Alpaca 美股模拟交易接口，可以体验真实下单。

任务

在 notebook 中新建代码单元格，完成以下**三个部分**（Part A、Part B-1、Part B-2 缺一不可）：

- **Part A**：SimBroker 回测基准
- **Part B-1**：用 Alpaca 数据源做历史回测，对比数据源差异
- **Part B-2**：用 LiveBroker 提交真实订单到 Alpaca 模拟盘，展示真实成交

无 Alpaca 账号时提供替代方案。

要求

1. 阅读以下 oxq 模块的源码，了解接口的输入、输出和参数含义：

- o `oxq.contrib.alpaca.client` — `AlpacaClient`
 - `AlpacaClient(api_key=..., secret_key=..., paper=True)`
 - `get_account()` -- dict (含 `equity`, `status`)
 - `get_positions()` -- list[dict] (每项含 `symbol`, `qty`, `avg_entry_price`)
 - `get_order(order_id)` -- dict (含 `symbol`, `side`, `qty`, `status`)
- o `oxq.contrib.alpaca.market_data` — `AlpacaMarketDataProvider`
 - `AlpacaMarketDataProvider(api_key=..., secret_key=..., feed="iex")`
 - `get_bars(symbol, start, end)` -- DataFrame
 - `get_bars_multi(symbols, start, end)` -- dict[str, DataFrame]
- o `oxq.trade.live_broker` — `LiveBroker`
 - `LiveBroker(api_key=..., secret_key=..., paper=True)`
 - `submit_order(order)` -- str (Alpaca 订单 ID)
 - `get_fills()` -- list[Fill]
 - `close()` — 关闭 WebSocket 和 HTTP 连接
- o `oxq.core.engine` — `Engine`
 - `Engine().setup(strategy, market=..., broker=..., start=..., end=...)` — 初始化但不运行
 - `Engine.dates` — 迭代用的日期列表
 - `Engine.step(date)` — 执行单个 bar
 - `Engine.result` — 获取结果
- o `oxq.trade.order_generator` — `generate_orders()`
- o `oxq.core.types` — `Position`
- o 策略相关： `make_us_strategy` 和 `make_us_rules` 辅助函数、 `Threshold` (信号)、 `RiskParityOptimizer` (组合优化器)、 `RebalanceFrequencyRule`、 `StopLossRule`

2. Part A——SimBroker 回测基准：

- o 下载美股数据：SPY、QQQ、GLD，起始日期 2021-01-01，用 `YFinanceDownloader` 下载
- o 构造与 Step 2 相同的策略和规则：用 `make_us_strategy(name)` 构造美股策略，用 `make_us_rules()` 构造美股规则
- o 用收盘价模式 + 佣金跑一次完整回测，`Engine.run()` 传入 `rules=make_us_rules()`，赋给 `result_sim`
- o 打印累计收益和交易笔数

3. 定义公共画图函数 `plot_sim_vs_live(norm_sim, norm_live, title, label_sim, label_live)`：

- o `figsize 12x8`，上下两个子图，高度比 3:1，`layout='constrained'`
- o 上图：两条归一化净值曲线（起点=100）
- o 下图：差距填充图（`fill_between`）

4. 检测 Alpaca 环境变量：用 `os.environ.get("ALPACA_API_KEY")` 判断。全部 Alpaca 操作包在 try/except 中，失败时自动切换到要求 7 的替代方案。

5. Part B-1——数据源差异（有 Alpaca 时执行）：

- o 用 `AlpacaMarketDataProvider(feed="iex")` 获取同期历史数据
- o 用 `Engine.setup() + Engine.step()` 逐 bar 执行，配合 `SimBroker`
- o 对比 YFinance 数据和 Alpaca IEX 数据的回测结果
- o 调用 `plot_sim_vs_live()` 画净值对比图，标题“同一策略，不同数据源”
- o 打印两个数据源的累计收益和差异

6. Part B-2——LiveBroker 实盘演示（有 Alpaca 时，紧接 B-1 执行，不可跳过）：

- 打印分隔线和标题"LiveBroker 实盘演示：提交真实订单到 Alpaca 模拟盘"
 - 用 `AlpacaClient.get_positions()` 获取当前持仓，打印每个标的的股数和成本价，将持仓转为 `dict[str, Position]`
 - 用 `AlpacaClient.get_account()` 获取账户权益
 - 用 `AlpacaMarketDataProvider` 获取各标的最新价格（如果当天无数据，用最近一个交易日的收盘价）
 - 用 `generate_orders(target_weights, positions, prices, total_capital=equity, lot_size=1)` 生成调仓订单
 - 如果无需调仓（`planned` 为空），打印"无需调仓，当前持仓已符合目标权重"
 - 如果有订单，用 `LiveBroker(paper=True)` 逐笔提交，打印每笔订单的提交确认
 - 等待 3 秒后调用 `live_broker.get_fills()` 获取成交回报
 - 打印成交表格：标的、方向、股数、成交价、回测价（最新收盘价）、滑点
 - 如果没有成交（市场关闭），用 `AlpacaClient.get_order(oid)` 查询每笔订单状态并打印
 - 调用 `live_broker.close()` 关闭连接
 - 用 `AlpacaClient.get_positions()` 查看最终持仓并打印
 - 将 Part B-1 的回测结果赋给 `result_live`（供 Step 4 使用）
7. 无 Alpaca 替代方案（无环境变量或 Alpaca 连接失败时执行）：

- 用 `SimBroker(fee_model=..., slippage_model=..., fill_price_mode=FillPriceMode.NEXT_OPEN)` 模拟实盘
- 调用 `plot_sim_vs_live()` 画回测 vs 模拟实盘的净值对比图
- 打印执行落差（implementation shortfall）
- 将模拟结果赋给 `result_live`（供 Step 4 使用）

验证

执行成功的标志：

- 单元格无报错运行完毕
- 有 Alpaca 时，输出必须包含以下**全部三部分**：
 1. SimBroker 回测基准的累计收益
 2. 数据源对比图 + Alpaca IEX 回测累计收益
 3. "LiveBroker 实盘演示"标题 + 当前持仓 + 调仓计划（或"无需调仓"）+ 最终持仓
- 无 Alpaca 时：自动切换到替代方案，显示模拟实盘的净值对比图
- `result_live` 变量被赋值，供 Step 4 使用
- 不包含任何硬编码的 API Key

Spec: 怎么知道执行得对不对？——执行报告

所有命令在沙箱外运行。

上下文

在 `q7-execution.ipynb` 中已有：

- Step 1 的订单生成实验（A 股 ETF）
- Step 2 的成交价压力测试和成本层叠（A 股 ETF）
- Step 3 的回测 vs 实盘对比（美股 ETF + Alpaca），`result_sim` 和 `result_live` 两个回测结果

Step 3 让学员看到了整体的执行落差。但"整体差 2%"不够——需要知道具体哪笔交易差得最多，是标的问题还是日期问题，大部分交易的执行质量如何。

任务

在 notebook 中新建代码单元格，用 `ExecutionReport` 逐笔对比回测和实盘的成交，分析执行质量分布。

要求

1. 阅读以下 oxq 模块的源码，了解接口的输入、输出和参数含义：

- `oxq.portfolio.execution_report` — `ExecutionReport` 和 `FillComparison`
 - `ExecutionReport(sim_fills=list[Fill], live_fills=list[Fill])`
 - `comparisons` 属性 -- `list[FillComparison]`
 - `summary()` -- dict, 含 `total_trades`, `matched_trades`, `sim_only_trades`, `live_only_trades`, `avg_price_slippage`, `total_fee_diff`
- `FillComparison` 属性：`symbol`, `date`, `side`, `sim_shares`, `sim_avg_price`, `sim_fee`, `live_shares`, `live_avg_price`, `live_fee`, `shares_diff`, `price_slippage`, `fee_diff`

2. 生成执行报告：

- 用 `result_sim.trades` 和 `result_live.trades` 构造 `ExecutionReport`
- 打印前 20 笔逐笔对比表：日期、标的、方向、回测价、实盘价、滑点、回测股数、实盘股数、股数差

3. 打印汇总统计：

- 总交易笔数、匹配交易数、仅回测有、仅实盘有
- 平均价格滑点、总手续费差异

4. 找出"最贵的一笔"：

- 从匹配交易（回测和实盘都有成交的）中，按 `price_slippage` 绝对值排序
- 打印最贵一笔的详情：日期、标的、方向、回测价、实盘价、滑点
- 如果滑点超过 1%，分析可能原因（波动大、流动性差、跳空）

5. 画执行质量分布图（`figsize 14×5`，两个子图并排）：

- 左图：滑点直方图（30 bins），标注零滑点线（红色虚线）和平均值线（橙色虚线）
- 右图：按标的分组的滑点箱线图（SPY/QQQ/GLD 各一个箱子，用不同颜色填充）

6. 打印执行质量统计：

- 总笔数、平均滑点、中位数滑点、最小/最大滑点
- 滑点超过 0.5% 的交易笔数和占比
- 总结：大部分交易正常，少数几笔偏差大——执行层面的"尾部风险"

7. 桥接 A 股：

- 美股模拟盘上的问题在 A 股一样存在：取整偏差（100 股整手更大）、价格滑点、交易成本（额外印花税）、执行落差（可能更大）
- 操作建议：用 `generate_orders()` 生成订单清单、定期对比净值监控落差、发现异常检查原因

验证

执行成功的标志：

- 单元格无报错运行完毕
- 逐笔对比表至少显示 20 行（或全部，取较小值）
- 汇总统计包含匹配交易数和平均滑点
- "最贵的一笔"有具体数值
- 直方图和箱线图正常渲染
- 桥接 A 股的操作建议完整

Spec: 策略是不是坏了？——监控仪表盘

所有命令在沙箱外运行。

上下文

学员跑完 Q7，策略终于在实盘运行了。但上个月赚了 3%，这个月亏了 5%——是正常波动还是策略出了问题？需要一个"仪表盘"，用具体指标替代焦虑。

任务

在 notebook `q8-iteration.ipynb` 中创建代码，跑基准回测后用 `StrategyMonitor` 构建监控仪表盘，画出三个滚动指标的时间序列图，并自动检测恶化时段。

要求

1. 获取当前工作目录，以此为根目录操作。

2. 阅读以下 oxq 模块的源码，了解接口的输入、输出和参数含义：

- o oxq.core — Engine, Strategy (注意 hypothesis 和 objectives 字段)
- o oxq.data — YFinanceDownloader, LocalMarketDataProvider
- o oxq.indicators — RollingVolatility
- o oxq.signals — Threshold
- o oxq.portfolio.optimizers — RiskParityOptimizer
- o oxq.rules — RebalanceFrequencyRule, StopLossRule
- o oxq.trade — SimBroker, FillPriceMode, PercentageFee
- o oxq.universe — StaticUniverse
- o oxq.observe — StrategyMonitor (构造参数、rolling_sharpe / rolling_drawdown / rolling_excess 属性、bad_periods 属性、summary() 方法)
- o oxq.portfolio.analytics — RunResult (注意 benchmark_prices 字段)

3. 创建 notebook q8-iteration.ipynb，导入所需库并设置中文显示：

- o oxq 模块：Engine, Strategy, YFinanceDownloader, LocalMarketDataProvider, RollingVolatility, Threshold, RiskParityOptimizer, RebalanceFrequencyRule, StopLossRule, SimBroker, FillPriceMode, PercentageFee, PercentageSlippage, StaticUniverse, StrategyMonitor, MarketStateDetector, ExperimentLog
- o from decimal import Decimal
- o pandas, numpy, matplotlib.pyplot
- o 设置 matplotlib 中文显示支持 (macOS: Arial Unicode MS / STHeiti, Windows: SimHei, axes.unicode_minus = False)

4. 定义常量和策略：

- o A 股标的：SYMBOLS = ("510300.SS", "513100.SS", "518880.SS")
- o 中文名映射：沪深300ETF、纳指100ETF、黄金ETF
- o 起始日期：2021-01-01，结束日期：当前日期
- o 下载数据 (YFinanceDownloader，失败时 try/except 用本地缓存)
- o 定义 make_strategy(name, hypothesis="", objectives=None) 工具函数：创建 Threshold 信号，设置 required_indicators，使用 RiskParityOptimizer(volatility_col="vol")
- o 定义 make_rules(freq=None, stop_loss=None) 工具函数：返回 [RebalanceFrequencyRule(interval_days=freq or BEST_FREQ), StopLossRule(threshold=stop_loss or BEST_SL)]
- o 佣金模型：PercentageFee(rate=Decimal("0.0006"), min_fee=Decimal("5"))

5. 跑基准回测：

- o 用 SimBroker(fee_model=..., fill_price_mode=FillPriceMode.NEXT_OPEN) 跑完整回测，Engine.run() 传入 rules=make_rules()
- o 回测后把沪深300价格放入 result_base.benchmark_prices["510300.SS"]，供 StrategyMonitor 使用
- o 打印累计收益、年化收益、最大回撤、夏普比率、交易笔数

6. 构建监控仪表盘：

- o 提取净值曲线 equity = pd.Series(dict(result_base.equity_curve))，计算 daily_ret (后续 step 会用到这两个变量)
- o 用 StrategyMonitor(result_base, benchmark="510300.SS", roll_window=63) 获取滚动指标
- o 画四行子图 (figsize 14×16，共享 x 轴，layout='constrained')：
 - 第 1 行：净值曲线 (起点=100)
 - 第 2 行：滚动夏普 (63日)，红色填充 <0 区域，标注零线和夏普=1 线

- 第 3 行：滚动回撤，红色填充
- 第 4 行：相对沪深300超额收益（年化），红色填充 <0 区域

7. 自动检测恶化时段：

- 遍历 `monitor.bad_periods`，打印每段的起止日期、持续天数、平均夏普
- 打印过渡：2022 年集中出现恶化，原因是什么？

验证

执行成功的标志：

- 单元格无报错运行完毕
- 基准回测有明确的收益和夏普数值
- 四行仪表盘图正常渲染
- 恶化时段列表非空（2022 年应有多个时段）
- `equity`、`daily_ret`、`result_base`、`monitor` 变量已定义（后续 spec 使用）

Spec: 问题出在哪？——诊断三问

所有命令在沙箱外运行。

上下文

在 `q8-iteration.ipynb` 中已有：

- Step 1 的监控仪表盘（基准回测、滚动指标、恶化时段检测）
- 已定义的变

量：`result_base`、`equity`、`daily_ret`、`monitor`、`SYMBOLS`、`NAMES`、`START`、`today`、`market`、`make_strategy`、`make_rules`、`FEE_MODEL`、`BEST_FREQ`、`BEST_SL`

Step 1 发现了多段恶化时段（2022 年集中出现），现在需要搞清楚原因。由外到内排查三层：执行 -- 市场 -- 策略。

任务

在 notebook 中新建代码单元格，完成三步诊断：排查执行落差、检测市场状态变化、检查参数稳定性。

要求

1. 阅读以下 oxq 模块的源码，了解接口的输入、输出和参数含义：

- o `oxq.trade` — `PercentageSlippage(rate)`
- o `oxq.observe` — `MarketStateDetector`（构造参数、`market_vol` / `states` / `high_vol_mask` / `vol_median` / `high_vol_line` 属性、`performance_by_state()` 方法）

2. 诊断 1——执行落差排查：

- o 跑"理想回测"（收盘价 + 佣金，无滑点）和"模拟实盘"（次日开盘 + 佣金 + 千分之一滑点），`Engine.run()` 均传入 `rules=make_rules()`
- o 计算两条净值曲线的滚动执行落差（63 日滚动均值）
- o 画两行子图（`figsize 14×7`）：上图两条净值曲线，下图滚动执行落差
- o 打印执行落差的累计值和每年均值
- o 把落差放到学员能理解的语境中：策略 N 年赚了 X%，其中 Y% 被执行成本吃掉，相当于每年 Z%
- o 强调重点是走势是否平稳（没有突然放大），而不是绝对数值
- o 打印结论：排除执行问题

3. 诊断 2——市场状态检测：

- o 用 `MarketStateDetector(result_base)` 检测市场状态
- o 打印波动率中位数、高波动阈值、高波动天数
- o 用 `detector.performance_by_state(result_base)` 分状态打印策略表现（年化收益、夏普、天数）
- o 画两行子图（`figsize 14×10`）：
 - 上图：净值曲线 + 高波动时段红色阴影叠加
 - 下图：市场波动率时间序列 + 中位数线 + 高波动阈值线
- o 打印结论：恶化时段和高波动高度吻合
- o 确保 `detector`、`high_vol_mask`、`market_vol`、`HIGH_VOL_LINE` 变量已定义（后续 spec 使用）

4. 诊断 3——参数稳定性检查：

- o 把数据分成前后两半（按交易日中点切分）
- o 对 5 种调仓频率（10/15/21/42/63 天）分别在两半上跑回测，`Engine.run()` 传入 `rules=make_rules(freq=频率)`
- o 打印对比表：频率、前半段年化、前半段夏普、后半段年化、后半段夏普
- o 打印两半各自的夏普排序
- o 打印结论：不同时段收益差异大，但参数排序是否一致

验证

执行成功的标志：

- 单元格无报错运行完毕
- 诊断 1：两条净值曲线 + 执行落差图正常渲染，结论为"排除执行问题"
- 诊断 2：市场状态图正常渲染，高波动阴影和恶化时段吻合
- 诊断 3：参数对比表有 5 行，两半段的夏普排序已打印
- `detector`、`high_vol_mask` 变量已定义（供 Step 3 使用）

Spec: 怎么改？——假设驱动的对照实验

所有命令在沙箱外运行。

上下文

在 `q8-iteration.ipynb` 中已有：

- Step 1 的监控仪表盘（恶化时段检测）
- Step 2 的三步诊断（执行 OK、市场状态变了、参数有改进空间）
- 已定义的变

量：`result_base`、`equity`、`daily_ret`、`detector`、`high_vol_mask`、`market_vol`、`SYMBOLS`、`NAMES`、`START`、`today`、`market`、`make_strategy`、`make_rules`、`FEE_MODEL`、`BEST_FREQ`、`BEST_SL`

诊断结论：策略对高波动的市场状态没有应对方案。现在用假设驱动的对照实验来迭代改进。

任务

在 notebook 中新建代码单元格，完成两轮迭代实验（调仓频率 + 波动率过滤），使用 Strategy 的 hypothesis / objectives 字段展示完整的实验 workflow，最后用 ExperimentLog 记录迭代历史。

要求

1. 阅读以下 oxq 模块的源码，了解接口的输入、输出和参数含义：

- o `oxq.core.strategy` — Strategy 的 `hypothesis: str` 和 `objectives: dict` 字段
- o `oxq.core.engine` — `Engine.run()` 方法（注意 `rules` 参数）
- o `oxq.portfolio.optimizers` — `RiskParityOptimizer`、`VolFilteredOptimizer`（包装器，高波动期将权重缩放 0.5）
- o `oxq.observe` — `ExperimentLog`（`add()` 方法、`to_dataframe()` 方法）

2. 迭代 1——调仓频率对照实验：

- o 创建 Strategy 时填写 `hypothesis="缩短调仓频率能改善高波动期表现"` 和对应的 `objectives`
- o 打印策略的 `hypothesis` 和 `objectives` 字段值
- o 对 6 种频率（5/10/15/21/42/63 天）分别跑回测，`Engine.run()` 传入 `rules=make_rules(freq=频率)`
- o 对每种频率，计算高波动时段的夏普（用 `high_vol_mask` 筛选对应日期的日收益）
- o 打印全时段指标表：频率、年化收益、最大回撤、夏普比率、高波动夏普，21 天行标注"-- 基准"
- o 画净值对比图（`figsize 14×7`，6 条线，21 天加粗），图例标注"高波动夏普"而非全时段夏普
- o 假设验证：先打印引导（"要看最后一列高波动夏普，不是倒数第二列全时段夏普"），再对比基准和最优频率的高波动夏普，判断假设确认还是推翻
- o 保留 `iter1_results` 变量

3. 迭代 2——波动率过滤对照实验：

- o 创建 Strategy 时填写 `hypothesis` 和 `objectives`（目标：回撤改善 $\geq 30\%$ ，夏普 ≥ 1.0 ）
- o 打印策略的 `hypothesis` 和 `objectives` 字段值
- o 用 `VolFilteredOptimizer` 包装 `RiskParityOptimizer`：高波动期自动将权重缩放 0.5，无需手动修改信号
- o 市场波动率使用 `detector.market_vol`（复用 Step 2 的 `MarketStateDetector`）
- o 测试 4 种配置：无过滤（基准 `RiskParityOptimizer`）、阈值 10%、阈值 15%、阈值 20%（后三者使用 `VolFilteredOptimizer`）
- o `Engine.run()` 传入 `rules=make_rules()`
- o 打印指标表：配置、年化收益、最大回撤、夏普比率、回撤改善比例
- o 画净值对比图（`figsize 14×7`，4 条线）
- o 假设验证：检查最佳配置是否满足两个验证标准，判断假设确认还是推翻
- o 打印 Trade-off 分析：年化收益降低了多少换来了多少回撤改善
- o 保留 `iter2_results`、`best_label`、`best_improve`、`best_freq_hv`、`best_hv_sharpe`、`dd_limit` 变量

4. 迭代记录表：

- o 用 `ExperimentLog(name="Q8 迭代实验")` 创建记录
- o 用 `log.add()` 添加两轮迭代的记录（`name`、`observation`、`hypothesis`、`criteria`、`result`、`conclusion`、`notes`）
- o 用 `log.to_dataframe().to_string(index=False)` 打印记录表
- o 打印过渡：这张表就是你的迭代 SOP

验证

执行成功的标志：

- 单元格无报错运行完毕
- 迭代 1：6 种频率指标表完整，净值图已保存，假设验证结论已打印
- 迭代 2：4 种配置指标表完整，净值图已保存，假设验证结论已打印
- 迭代记录表包含 2 行（ iter1 rejected + iter2 confirmed ）
- Strategy 的 hypothesis 和 objectives 字段值在输出中可见

Spec: 从直觉到因子——回顾、评估方法与 GDP 因子实验

所有命令在沙箱外运行。

上下文

这是 Q9 章节的第一个 spec。学员已经完成 Q2-Q8，用 GDP 选国家、用动量排名买卖，但还不知道"因子"这个概念。本 spec 对应 notebook `q9-daily-work.ipynb` 的 Step 1（Cell 0-10），从零引入因子概念和 IC 评估方法，并用 GDP 数据做第一个因子评估实验。

任务

在 notebook `q9-daily-work.ipynb` 中创建 Step 1 的所有单元格：导入库、回顾因子表格、IC 方法说明、GDP 因子 IC 实验、排名对比图、GDP 分析、因子分类全景表。

要求

1. 阅读以下 oxq 模块的源码，了解接口的输入、输出和参数含义：

- `oxq.data.factors.WorldBankFetcher`
- `oxq.data.factors.FactorDownloader`
- `oxq.data.factors.read_factor`
- `oxq.data.loaders.YFinanceDownloader`
- `oxq.data.market.LocalMarketDataProvider`
- `oxq.indicators.momentum.Momentum`
- `oxq.indicators.rolling_volatility.RollingVolatility`

2. 创建章节标题 markdown 单元格（“量化的日常是什么？”），说明前八章从零搭建了完整策略，这一章回头看这些决策背后的核心工作——因子研究。

3. 创建 Step 1 标题 markdown（“从直觉到因子”），包含因子回顾表格：

- Q2 选什么 → GDP 总量/增速 → GDP 因子 → 宏观因子
- Q3 买多少 → 风险调整动量 → 动量因子 → 动量因子

4. 创建 IC 方法说明 markdown：解释 Spearman IC 的含义（因子排名 vs 收益排名的相似度）、IC 均值的参考阈值（0.03 / 0.05 / 0.1）、ICIR 的概念。

5. 创建代码单元格：导入库并设置中文字体。需要导入：

- `warnings`（并 `filterwarnings("ignore")`）
- `matplotlib.pyplot`, `numpy`, `pandas`, `scipy.stats`
- `oxq.data.loaders.YFinanceDownloader`
- `oxq.data.market.LocalMarketDataProvider`
- `oxq.data.factors.WorldBankFetcher`, `FactorDownloader`, `read_factor`
- `oxq.indicators.momentum.Momentum`
- `oxq.indicators.rolling_volatility.RollingVolatility`
- 设置 `plt.rcParams['font.sans-serif']` 和 `axes.unicode_minus`
- 开启 `%matplotlib inline`

6. 创建代码单元格：下载 10 个国家的 GDP 数据。

- 国家列表：USA, CHN, JPN, DEU, GBR, FRA, IND, BRA, CAN, KOR
- 使用 `WorldBankFetcher()` + `FactorDownloader(fetcher, sub="macro")`
- 调用 `dl.download("gdp", start="2010", end="2024", countries=countries)`
- 用 `read_factor("gdp", ...)` 读取，打印形状
- 以万亿美元为单位显示（除以 `1e12`），设置 `columns.name="国家"`、`index.name="年份"`

7. 创建代码单元格：下载 10 个国家对应 ETF 的价格数据。

- 映射关系：USA → QQQ, CHN → FXI, JPN → EWJ, DEU → EWG, GBR → EWU, FRA → EWQ, IND → INDA, BRA → EWZ, CAN → EWC, KOR → EWY
- 使用 `YFinanceDownloader()` 下载，时间范围 2011-01-01 至 2026-12-31
- 使用 `LocalMarketDataProvider().get_bars()` 读取
- 打印覆盖表（国家、ETF、数据条数、起始/结束日期）

8. 创建代码单元格：计算两个 GDP 因子的年度 IC 并画图。

- **GDP 总量因子**：当年 GDP 排名 → 预测下一年 ETF 收益排名
- **GDP 增速因子**：当年 GDP 增速排名（`gdp_df.pct_change()`）→ 预测下一年 ETF 收益排名
- ETF 年度收益率：年末收盘价 / 年初收盘价 - 1
- IC 计算：对每个“因子年 → 收益年”配对，用 `scipy.stats.spearmanr` 计算相关系数
- 画 1×2 并排柱状图（`figsize 14×5`），正相关蓝色、负相关红色，加 IC 均值虚线

- 打印两个因子的 IC 均值和 ICIR

9. 创建代码单元格：IC 排名对比图（slope chart）。

- 选 GDP 总量因子 IC 最大的那一年
- 画因子排名 vs 收益排名的连线图（figsize 8×6）
- 中美两国用红色粗线高亮，其他灰色细线
- 左侧标注 GDP 排名，右侧标注收益排名
- 中文国名映射

10. 创建 GDP 因子分析 markdown：GDP 总量因子 IC 均值为正，呼应 Q2 选中美的逻辑；GDP 增速因子更差；局限性（年度数据、小样本）。

11. 创建因子分类全景表 markdown，6 类因子：

- 价值、动量、质量、波动率、宏观、另类
- 每类一句话含义、常见例子、数据来源

12. 创建过渡 markdown：GDP 因子受限于年度频率和小样本，接下来用日频价格数据和更多标的评估动量因子。

结果呈现

- 1. 库导入成功提示
- 2. GDP 数据表（万亿美元）
- 3. 10 国 ETF 覆盖表
- 4. 1×2 GDP 因子 IC 柱状图（总量 vs 增速）
- 5. IC 均值和 ICIR 打印
- 6. 排名对比 slope chart（最佳 IC 年份）
- 7. GDP 分析 markdown
- 8. 因子分类全景表
- 9. 过渡文案

验证

- 所有单元格无报错运行完毕
- GDP 数据包含 10 个国家、2010-2024 年
- 10 只 ETF 数据均有数百至数千条记录
- IC 柱状图显示约 10 个年度配对，正负均有
- slope chart 中美两国红色高亮可见
- 因子分类表有 6 行

Spec: 动量因子 IC 分析——找到问题

所有命令在沙箱外运行。

上下文

在 `q9-daily-work.ipynb` 的 Step 1 中，已完成库导入、中文字体设置、GDP 因子 IC 实验。已有的变量包括：`downloader`（`YFinanceDownloader`）、`provider`（`LocalMarketDataProvider`）。本 spec 对应 Step 2（Cell 11-16），将标的池从 10 只扩展到 25 只，用日频数据做动量因子的 IC 分析。

任务

在 notebook 中新建 Step 2 的所有单元格：构建 25 只标的池、计算动量因子与前向收益、IC 分析表和图、按波动率分组的 IC 分析、问题总结。

要求

1. 创建 Step 2 标题 markdown ("动量因子——涨得好的会继续涨吗?")。
2. 创建代码单元格：构建 25 只标的池并下载数据。
 - 22 个国家/地区 ETF + 3 个大宗商品 ETF，完整映射：
 - 原 10 国：USA → QQQ, CHN → FXI, JPN → EWJ, DEU → EWG, GBR → EWU, FRA → EWQ, IND → INDIA, BRA → EWZ, CAN → EWC, KOR → EWY
 - 新增 12 国：AUS → EWA, TWN → EWT, MEX → EWW, ITA → EWI, ESP → EWP, SWE → EWD, NLD → EWN, THA → THD, MYS → EWM, ZAF → EZA, ISR → EIS, SGP → EWS
 - 大宗商品：GOLD → GLD, OIL → USO, SILVER → SLV
 - 时间范围：2015-01-01 至 2025-12-31
 - 复用 Step 1 的 downloader 和 provider
 - 打印覆盖表（代号、Ticker、中文名、起始/结束日期、交易天数）
3. 创建代码单元格：计算两个动量因子和三个前向收益窗口。
 - 使用 `Momentum().compute(df, column="close", period=20)` 计算 20 日动量
 - 使用 `RollingVolatility().compute(df, column="close", period=20)` 计算 20 日波动率
 - RAM = 动量 / 波动率（风险调整动量）
 - 前向收益窗口 [10, 20, 40] 交易日：`df["close"].pct_change(h).shift(-h)`
 - 构建横截面矩阵：`mom_df`, `ram_df`, `vol_df`, `fwd_dfs`（dict by horizon）
 - 打印矩阵形状和示例
4. 创建代码单元格：IC 分析，2 因子 × 3 窗口 = 6 组。
 - 实现 `compute_ic_series(factor_df, fwd_df, min_assets=5)` 函数：对每个交易日，取因子值和前向收益的交集标的（至少 `min_assets` 个），计算 Spearman IC
 - 打印 IC 对比表（因子 → 前向窗口、IC 均值、IC 标准差、ICIR、IC>0 占比）
 - 画 2×3 子图（`figsize 18×8`）：每格画 IC 柱状图（`width=2, alpha=0.2, steelblue`）+ 60 日滚动均值曲线（`darkblue`）+ IC 均值红色虚线
 - 总标题"两个动量因子 × 三个前向收益窗口 IC 对比"
 - 保留 `ic_ts = ic_results["风险调整动量 - 20日"]` 和 `fwd_df = fwd_dfs[20]` 供后续使用
5. 创建代码单元格：按标的自身波动率分组 IC 分析。
 - 实现 `compute_grouped_ic(factor_df, fwd_df, vol_df, min_assets=6)` 函数：
 - 每个交易日，取当天各标的的 20 日波动率，按中位数分为高波动组和低波动组
 - 分别在组内计算 Spearman IC
 - 要求交集标的至少 12 个，每组至少 `min_assets` 个
 - 对两个因子（20 日动量、风险调整动量）分别做分组 IC
 - 打印分组 IC 表（因子、整体 IC、高波动组 IC、低波动组 IC、Spread）
 - 以 RAM 为例画分组 IC 时间序列对比图（`figsize 14×6`）：
 - 高波动标的 IC（红色，60 日滚动均值）
 - 低波动标的 IC（蓝色，60 日滚动均值）
 - 图例中标注各组 IC 均值
6. 创建问题总结 markdown ("动量因子的问题")：
 - 动量因子整体 IC 接近 0，截面预测能力很弱
 - 波动率是关键变量：高波动标的的动量 IC 明显差于低波动标的
 - RAM 在低波动组表现更好，但对高波动标的的依然无效
 - 结论：一个因子不够，需要补充能感知波动率环境的因子

结果呈现

- 1. 25 只标的覆盖表
- 2. 动量因子矩阵形状和示例数据
- 3. IC 对比表（6 组）
- 4. 2×3 IC 时间序列子图
- 5. 分组 IC 表
- 6. 分组 IC 时间序列对比图（高波动 vs 低波动）
- 7. 问题总结 markdown

验证

- 所有单元格无报错运行完毕
- 25 只标的全部加载成功
- IC 对比表有 6 行 (2 因子 $\times$ 3 窗口) , IC 均值绝对值在合理范围 (接近 0)
- 2 $\times$ 3 子图中每格都有 IC 柱状图和滚动均值线
- 分组 IC 表显示高波动组 IC 低于低波动组 (spread $>$ 0)
- 分组 IC 对比图红蓝两条线走势有明显差异

Spec: 从因子到策略——让数据说话

所有命令在沙箱外运行。

上下文

在 `q9-daily-work.ipynb` 的 Step 2 中，已完成 25 只标的的动量因子 IC 分析和分组分析。已有变量包括：`all_data`（25 只标的的数据 dict）、`vol_df`（波动率矩阵）、`fwd_dfs`（前向收益 dict）、`ic_results`（6 组 IC 时间序列）、`compute_ic_series` 函数、`downloader`、`OxqMomentum` / `OxqRollingVol` 等尚未导入。本 spec 对应 Step 3（Cell 17-24），补充因子、总结发现、改进策略并回测对比。

任务

在 notebook 中新建 Step 3 的所有单元格：补充因子 IC、因子评估发现、策略改进框架、Q3 标的回测、波动率过滤改进、对比分析。

要求

1. 创建 Step 3 标题 markdown ("从因子到策略——让数据说话")。
2. 创建代码单元格：补充因子 IC 计算。
 - **波动率因子**：复用 Step 2 中的 `vol_df`
 - **短期反转因子**：`-df["close"].pct_change(5)` (近 5 日收益取负)，构建 `rev_df` 矩阵
 - 用 `compute_ic_series` 计算波动率因子和反转因子的 IC (20 日前向窗口)
 - 复用 Step 2 的 RAM IC：`ic_results["风险调整动量 - 20日"]`
 - 打印三因子 IC 对比表 (因子、IC 均值、IC 标准差、ICIR、IC>0 占比)
3. 创建因子评估发现 markdown ("因子评估的发现")：
 - 三个因子各有侧重：动量看趋势、波动率看风险、反转看短期超跌
 - 关键发现：动量因子在高波动标的上 IC 为负，波动率是动量失效的主要原因
 - 因子多了需要机器学习来组合 (决策树/随机森林、XGBoost/LightGBM、神经网络)，这是未来方向
 - 当前先用最简单的方式——基于发现给策略加波动率过滤
4. 创建策略改进方向 markdown ("因子评估的发现能改进策略吗?")：
 - 两层选择框架：第一层 GDP 因子选候选池，第二层动量因子做择时
 - 因子评估告诉我们动量在高波动时容易出错，加波动率过滤应该能改善
5. 阅读以下 oxq 模块的源码，了解接口的输入、输出和参数含义：
 - `oxq.core.Engine`
 - `oxq.core.Strategy`
 - `oxq.indicators.Momentum` (注意：oxq 内部的，非 Step 2 用的独立版本)
 - `oxq.indicators.RollingVolatility` (同上)
 - `oxq.indicators.Ratio`
 - `oxq.portfolio.optimizers.TopNRankingOptimizer`
 - `oxq.signals.Threshold`
 - `oxq.trade.SimBroker`
 - `oxq.universe.StaticUniverse`
 - `oxq.rules.RebalanceFrequencyRule`
6. 创建代码单元格：下载 Q3 三只标的并运行基准回测。
 - 导入 oxq 组件：`Engine`，`Strategy`，`Momentum` as `OxqMomentum`，`RollingVolatility` as `OxqRollingVol`，`Ratio`，`TopNRankingOptimizer`，`Threshold`，`SimBroker`，`StaticUniverse`，`RebalanceFrequencyRule`，`LocalMarketDataProvider` as `OxqMarket`
 - Q3 标的：510300.SS (沪深300)、513100.SS (纳指100)、518880.SS (黄金)
 - 起始日期 2021-01-01，结束日期用 `pd.Timestamp.now().strftime("%Y-%m-%d")`
 - 下载数据并打印每只标的的交易天数
 - 基准策略配置：
 - `StaticUniverse(symbols=q3_symbols, name="q3-macro-etf")`
 - `Threshold` 信号，`required_indicators` 包含 `mom` (20日)、`vol` (20日)、`ram` (Ratio)
 - `TopNRankingOptimizer(score_col="ram", n=3, filter_negative=True)`
 - `RebalanceFrequencyRule(interval_days=21)`，`data_start="2020-06-01"`
 - 打印基准策略的累计收益、年化波动率、最大回撤、夏普比率
7. 创建代码单元格：波动率过滤改进。
 - 先计算市场平均波动率：三只标的的 20 日波动率均值 (从 2020-06-01 开始)
 - 实现 `VolFilteredOptimizer` 类，包装 `TopNRankingOptimizer`：
 - 接受 `base_optimizer`、`vol_threshold`、`market_vol` 参数

- `optimize` 方法：先调用 `base optimizer` 获取权重，再从 `indicators` 中取最新日期查 `market_vol`，若超过阈值则将所有持仓权重减半、多余部分分配给 CASH
 - 暴露 `name` 属性和 `required_indicators` 属性
 - 波动率阈值取市场波动率的中位数，打印阈值和高波动天数占比
 - 构建波动率过滤策略（信号配置与基准相同，`optimizer` 换为 `VolFilteredOptimizer`）
 - 运行回测，打印结果
8. 创建代码单元格：两策略对比。
- 打印指标对比表（策略、累计收益、年化波动率、最大回撤、夏普比率）
 - 画净值曲线对比图（`figsize 14×7`）：
 - 基准（蓝色 #3498DB）、波动率过滤（橙色 #E67E22）
 - 归一化到起点 = 100
 - 图例中标注各自夏普比率
 - 标题"因子评估 → 策略改进：基准 vs 波动率过滤"
 - 打印分析：对比夏普和回撤，若回撤改善则计算改善幅度
9. 创建小结 markdown：
- 两层选择框架（GDP 因子 → 动量因子）
 - 因子评估发现问题（高波动时动量 IC 为负）
 - 用发现改进策略（波动率过滤降回撤）
 - 量化日常循环：找因子 → 评估 → 发现问题 → 改进策略
 - 未来方向：更多因子、机器学习组合

结果呈现

- 1. 三因子 IC 对比表
- 2. 因子评估发现 markdown
- 3. 策略改进方向 markdown
- 4. Q3 三只标的覆盖信息
- 5. 基准策略回测指标
- 6. 波动率阈值和高波动天数占比
- 7. 波动率过滤策略回测指标
- 8. 指标对比表
- 9. 净值曲线对比图
- 10. 动态分析文本
- 11. 小结 markdown

验证

- 所有单元格无报错运行完毕
- 三因子 IC 对比表有 3 行，IC 值在合理范围
- Q3 三只标的的数据均有数百条以上记录
- 基准策略和波动率过滤策略均有有效回测结果（累计收益、夏普等非 NaN）
- 净值曲线图显示 2 条线（蓝色和橙色）
- 对比表清晰展示两种策略的差异